Java Full Stack Development

1. OAuth2 - OIDC Overview

Module Topics

- Why OAuth2 Exists
- OAuth2 Architecture
- The Four Roles
- Grant Types
- PKCE Protocol
- JWT Tokens
- Refresh Tokens

Why OAuth2 Exists

Anti-pattern

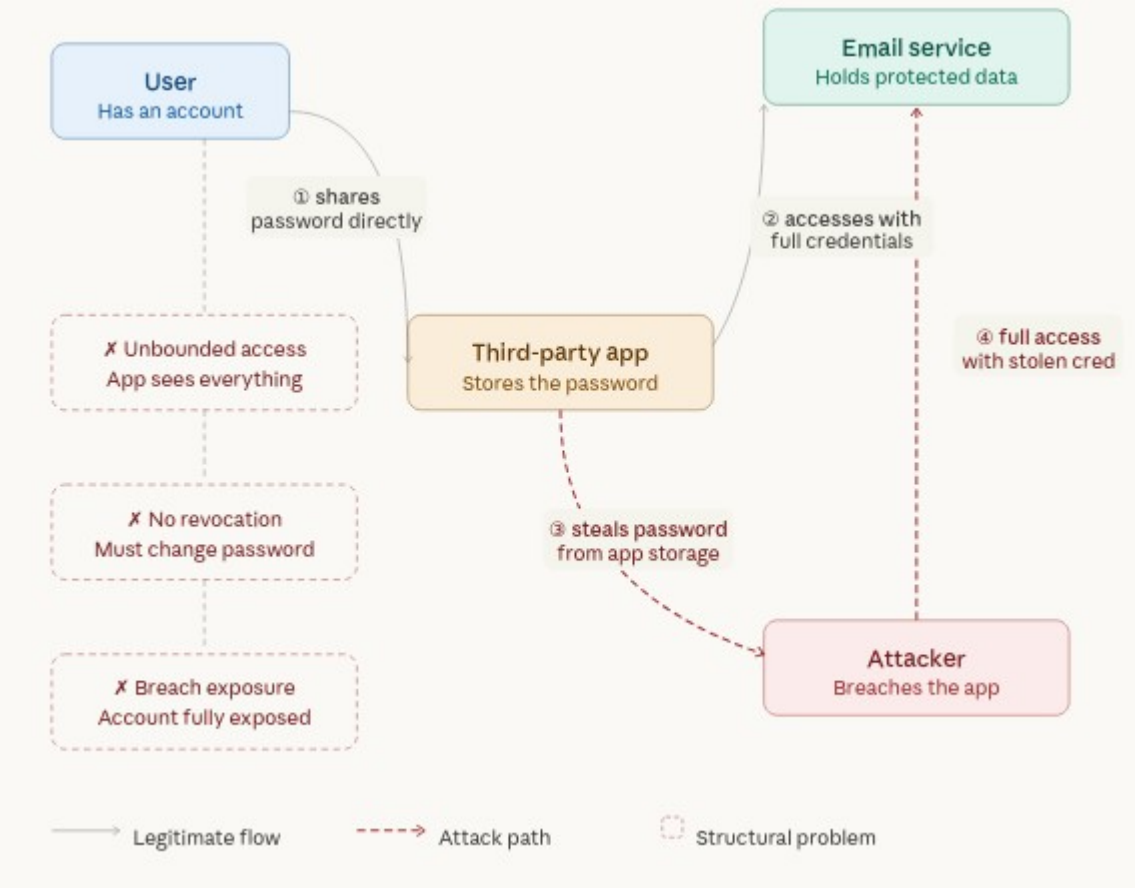
- The credential-sharing anti-pattern
  - Dominant pattern for third-party integrations was simple and dangerous
    - If you wanted an application to access your data, you gave it your password
    - The application stored that password, used it whenever it needed access, and retained it indefinitely
  - This was not a fringe behaviour
    - It was the standard pattern for consumer and enterprise integrations throughout the 2000s.
    - Every application followed the same mode.
  - The problems this create were structural
    - No amount of careful coding or process logic on the client or server side could fully address them.

Anti-pattern

- Problem 1: Unbounded access
  - The third-party application received a credential with full, unrestricted access to the application or account
    - Access was not fine grained
    - The access granted was usually far broader than the access actually required
- Problem 2: No selective revocation
  - When access was no longer needed, or when trust in the third party was lost
  - The only mechanism for revoking access was changing the password
  - Changing the password revoked access for all parts of the application simultaneously
    - Including the ones that were still trusted or necessary
  - There was no way to selectively restrict acces to some functionality and not other functionality

Anti-pattern

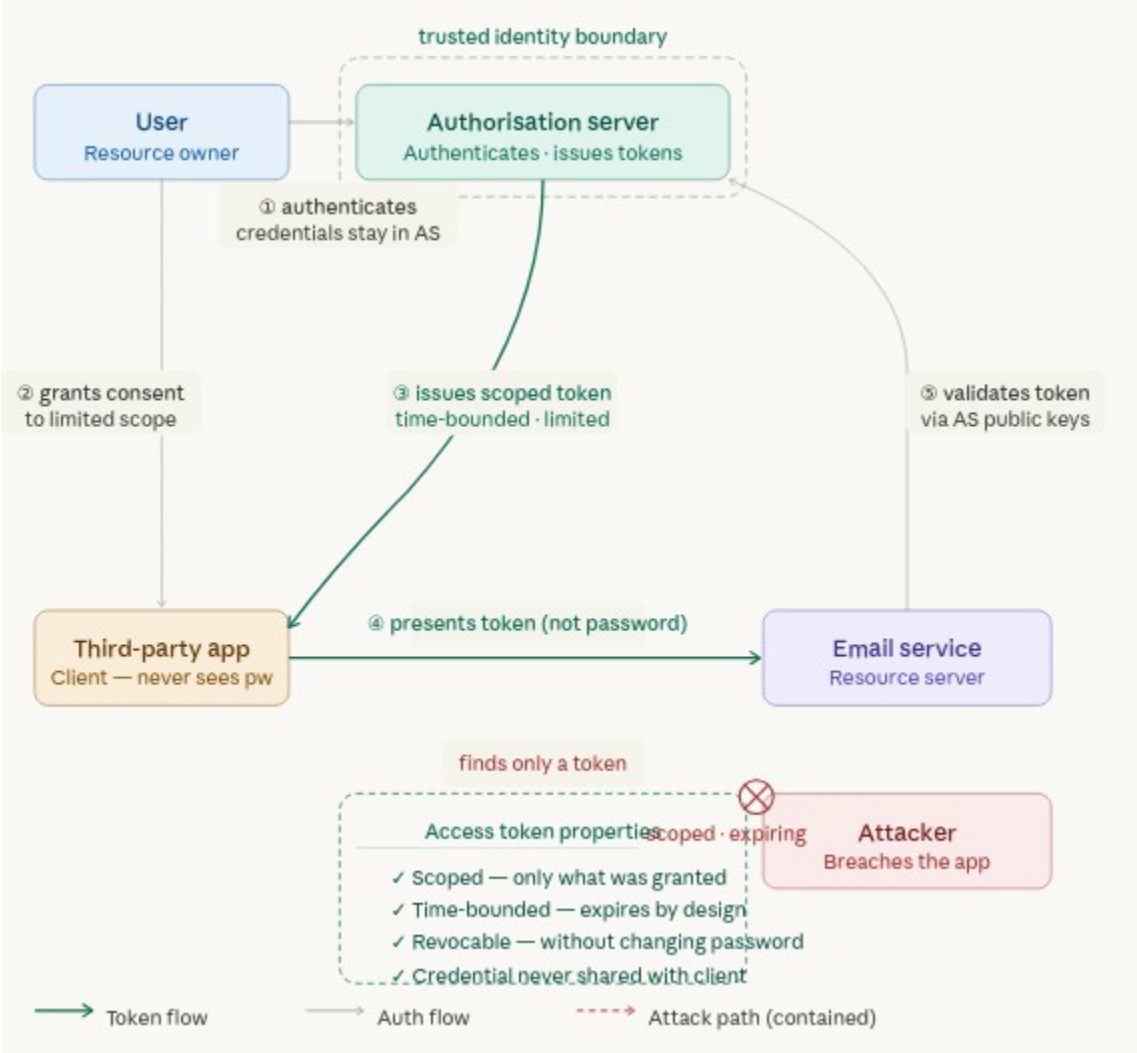
- Problem 3: Compounding breach exposure
  - Every application holding the credentials was a breach target
  - A breach of any one of them exposed the whole account completely
  - Ten integrations meant ten breach surfaces, each with full access
- Problem 4: No user visibility
  - Users had no mechanism to see what applications held their credentials
  - Or when those applications last used them, or what had done with them



OAuth2 Architecture

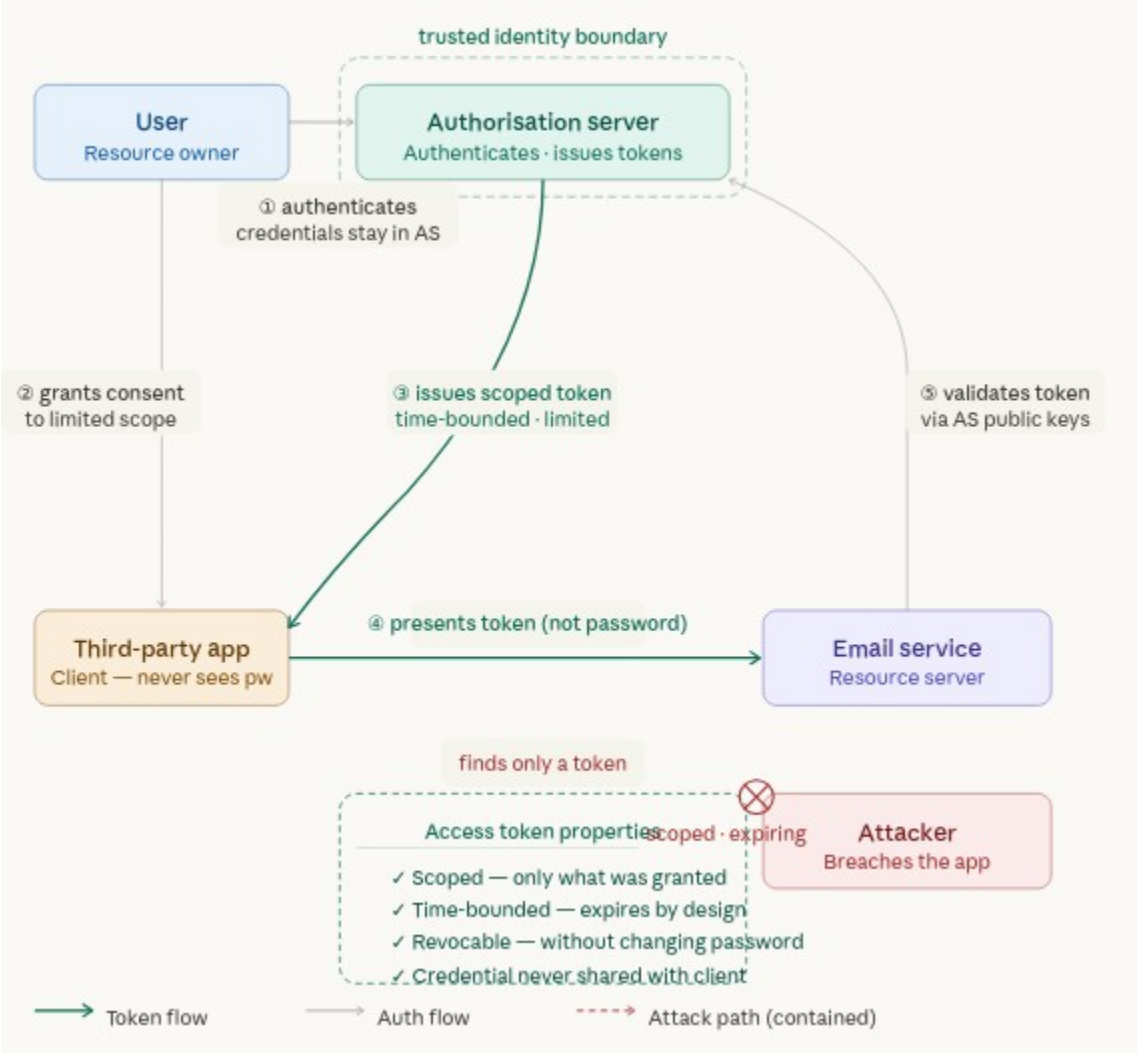
Delegated Authorization

- OAuth2 was designed specifically to eliminate the credential-sharing pattern
- Core innovation
  - Instead of sharing a credential that grants total access
  - The user authorities a limited, scoped, time-bounded token
  - The token grants access to exactly the resources and actions that are needed and nothing more



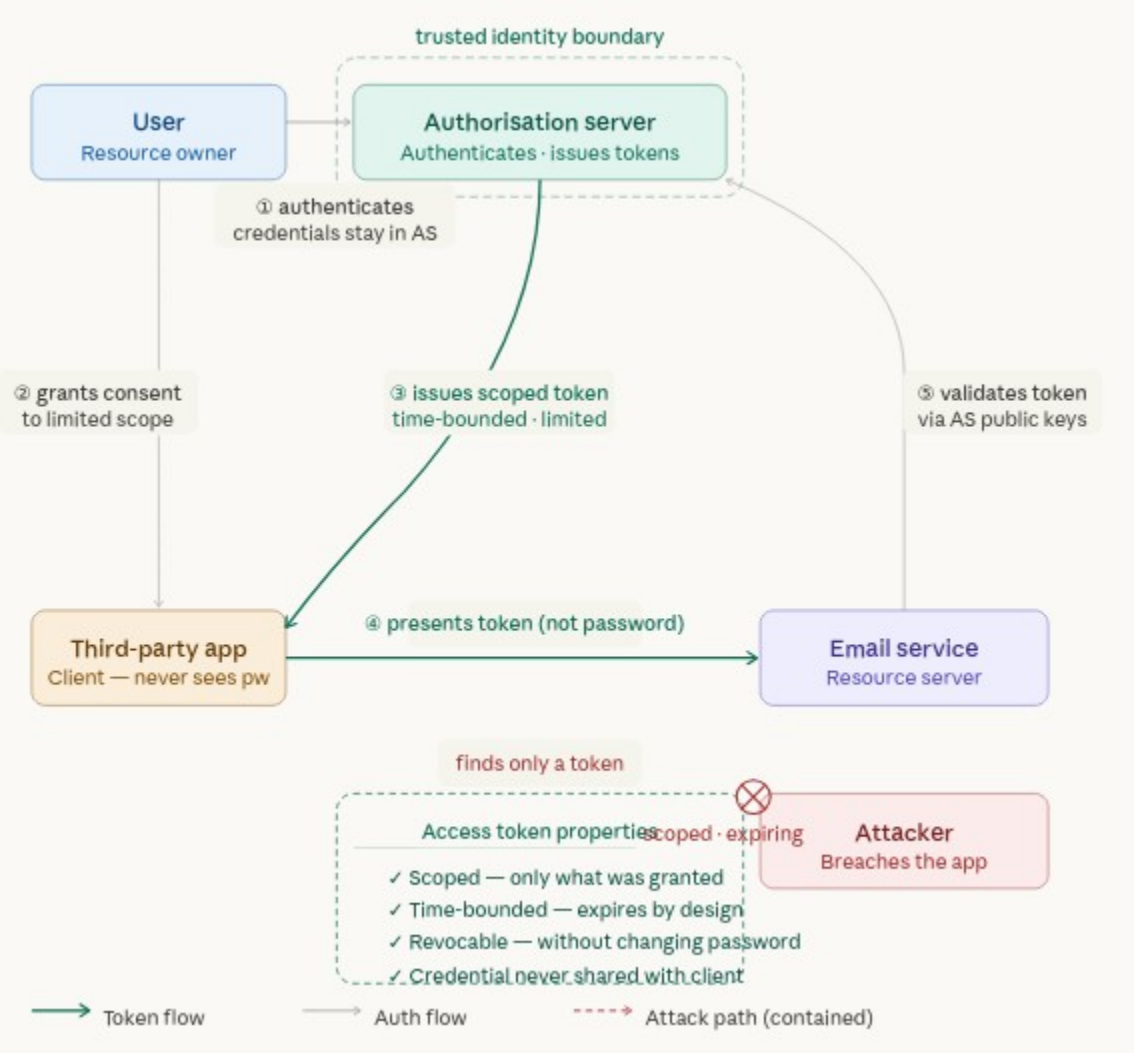
Delegated Authorization

- The three properties that make this work
- Credentials never leave the authorization server
  - The third-party application never sees the user's password
  - The user authenticates directly with the authorization server
  - The application receives only a token in return
  - The credentials are never in the application's possession



Delegated Authorization

- Tokens encode exactly what they permit
  - An access token carries its permissions within it
    - Which resources it covers
    - Which operations it allows
    - When it expires
    - Who it was issued for
  - An application that receives a read-only token cannot write
  - An application that receives a token scoped to the calendar cannot access email
  - The scope of access is defined at issuance and cannot be expanded by the application



Delegated Authorization

- Revocation is selective and immediate
  - Revoking a token affects only that token
  - Not any other application, not the user's password, not any other session
  - A user can revoke access to one application while every other integration continues working

Before OAuth2	After OAuth2
Full credential — password	Scoped token — limited permission
Unbounded access	Access bounded to declared scope
No expiry	Time-bounded — expires by design
Revoke by password change	Revoke by token invalidation
No visibility for user	Consent screen and token management
One breach exposes everything	Breach of token exposes only its scope

OAuth2

- OAuth2 Is an authorization framework
  - Not an authentication protocol
- What OAuth2 answers
  - What is this token allowed to do?
  - OAuth2 governs access to resources
  - It defines
    - How a client application obtains permission to act on behalf of a user
    - What that permission covers
    - How long it lasts
  - It says nothing about who the user is

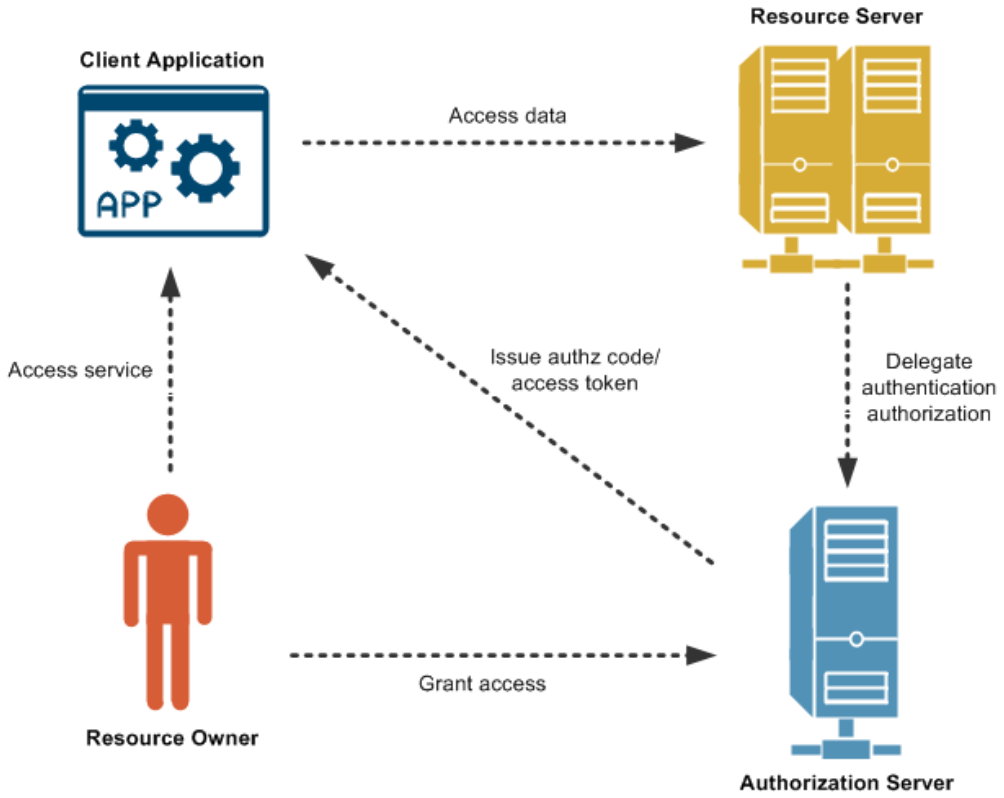


Image Credit: https://docs.oracle.com/cd/E50612_01/doc.11122/oauth_guide/content/oauth_intro.html

OAuth2

- What OAuth2 does not answer
  - Who is this user?
  - OAuth2 does not define how identity is asserted
  - A valid access token tells an application that someone authorized a token with a given scope
  - It does not, by itself, identify who that someone is

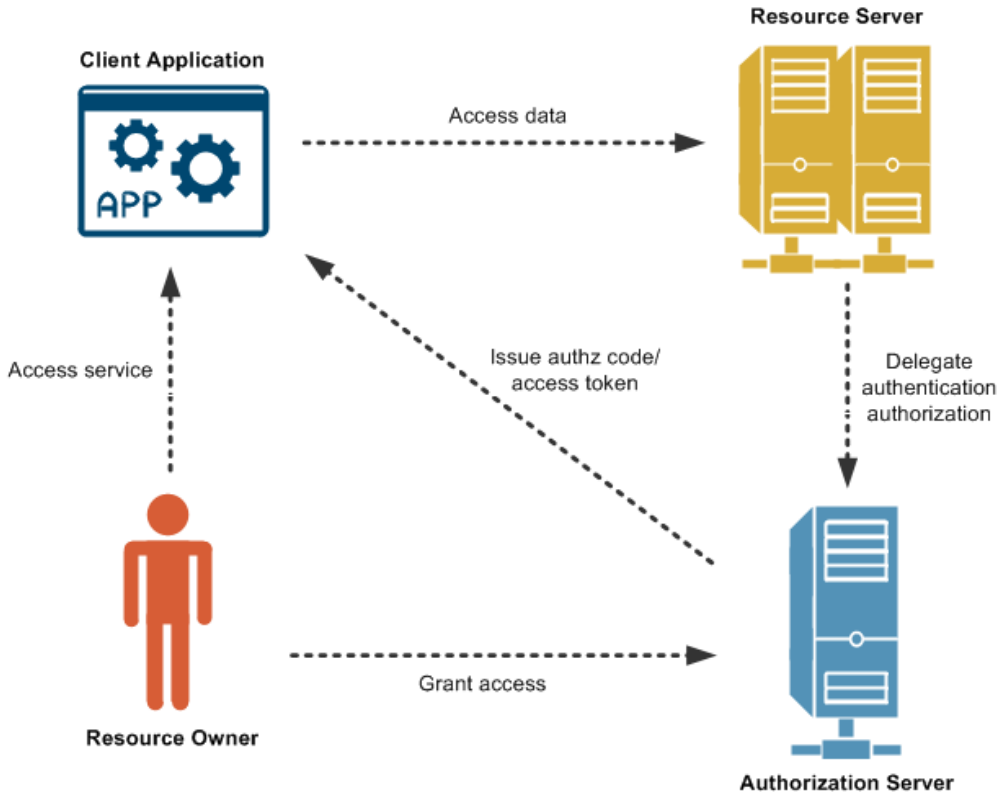


Image Credit: https://docs.oracle.com/cd/E50612_01/doc.11122/oauth_guide/content/oauth_intro.html

OpenID Connect

- OpenID Connect
  - OIDC is an identity layer built on top of OAuth2
- It adds the id_token
  - A JWT that carries verified identity claims about the authenticated user
    - Their unique identifier, name, email address, and the time and method of their authentication
  - OAuth2 says
    - *"This token is permitted to read employees"*
  - OIDC says
    - *"This token was issued to Jason Smith, who authenticated at 14:32 UTC using MFA."*

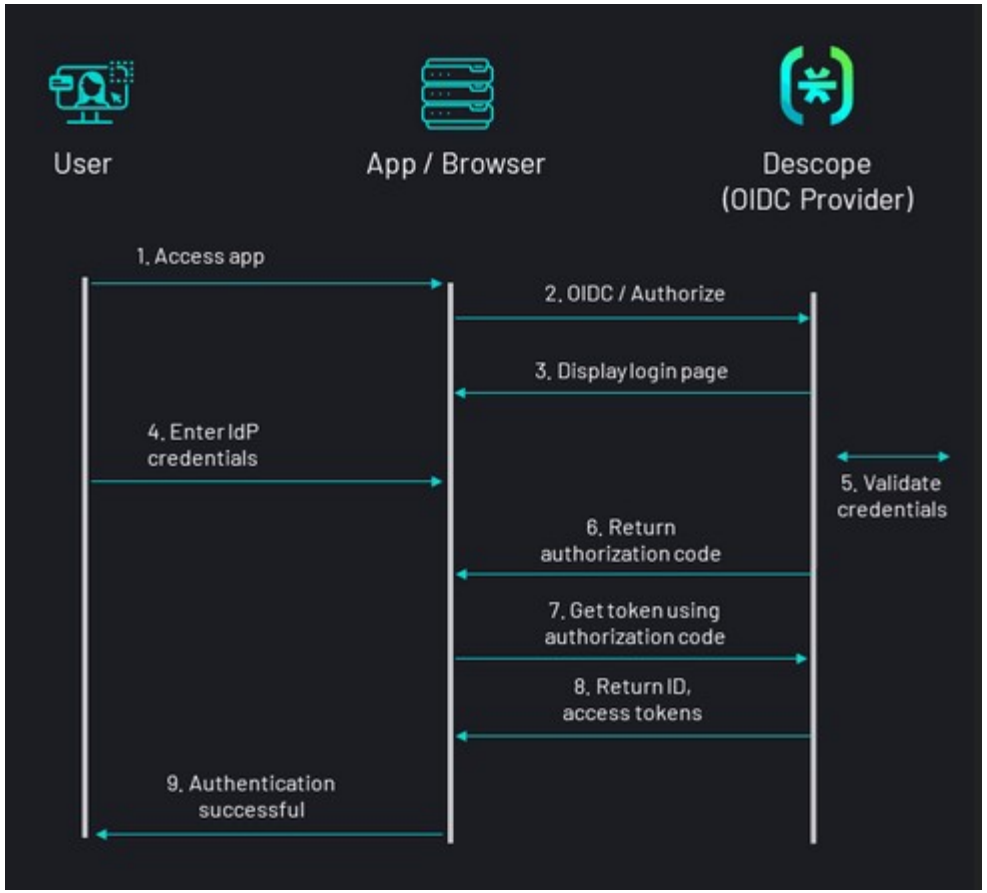


Image Credit: https://www.descope.com/learn/post/oidc

OAuth2 / OIDC

- IOAuth2 and OIDC are almost always deployed together
- Provides both dimensions of the security model
- OAuth2 authorizes what the client can do
  - Resource access, scopes, token issuance
- OIDC authenticates who the user is
  - Identity claims, authentication context, session management

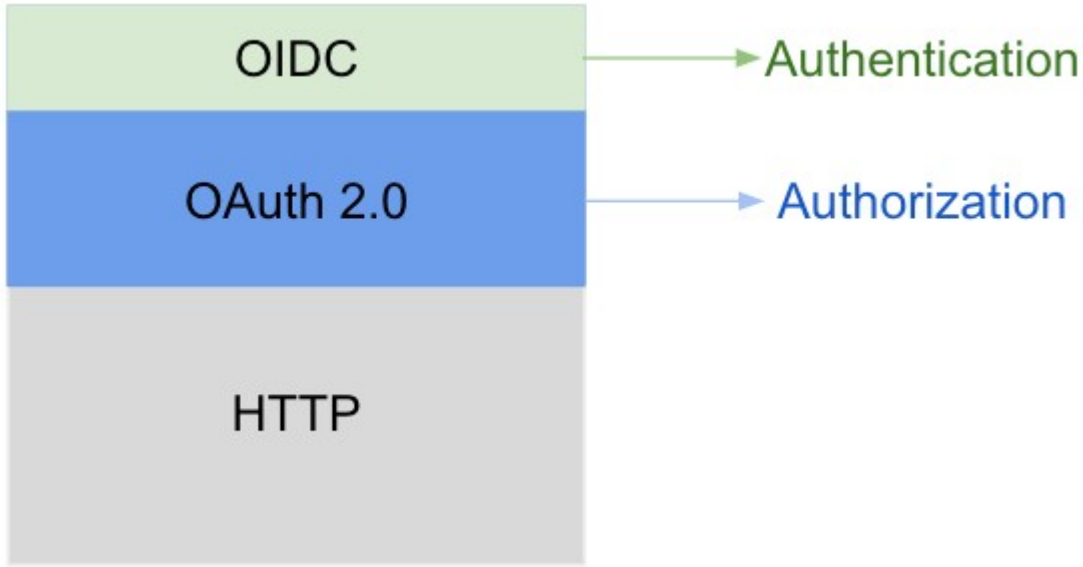


Image Credit:https://neungs-2.github.io/auth/oauth-oidc/

OAuth2 Limits

- OAuth2 is a delegation protocol
  - It solves the credential-sharing problem but it does not solve everything
- Does not define authorization logic
  - Whether a particular user should have a particular permission is an application-level decision
    - OAuth2 has no opinion about it.
  - A valid token with the correct scope tells the API that the client was granted that scope
    - It does not confirm whether the user behind the token is entitled to that access
    - That logic belongs in the client application

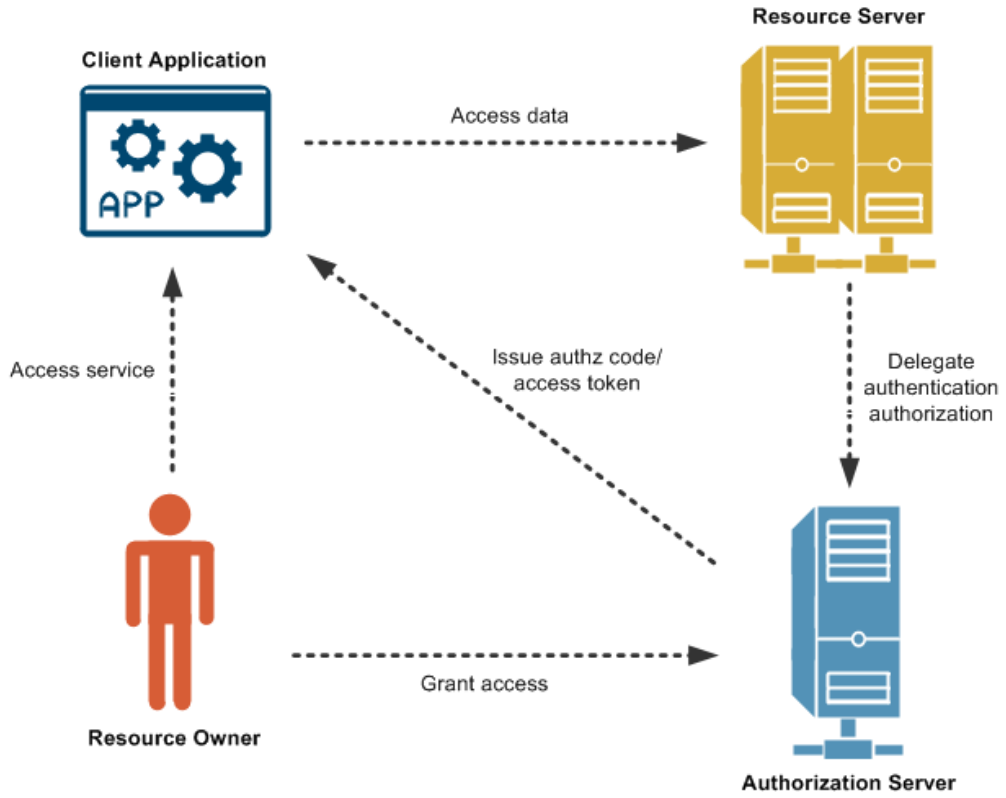


Image Credit: https://docs.oracle.com/cd/E50612_01/doc.11122/oauth_guide/content/oauth_intro.html

OAuth2 Limits

- Does not define secure token storage on the client
  - How an application stores the access token it receives is outside the scope of the specification
  - Browser-based applications that store tokens in localStorage expose them to cross-site scripting
  - Native applications that store tokens without platform-level encryption expose them to device compromise
  - The authorization server issues the token
  - Securing it is the application's responsibility

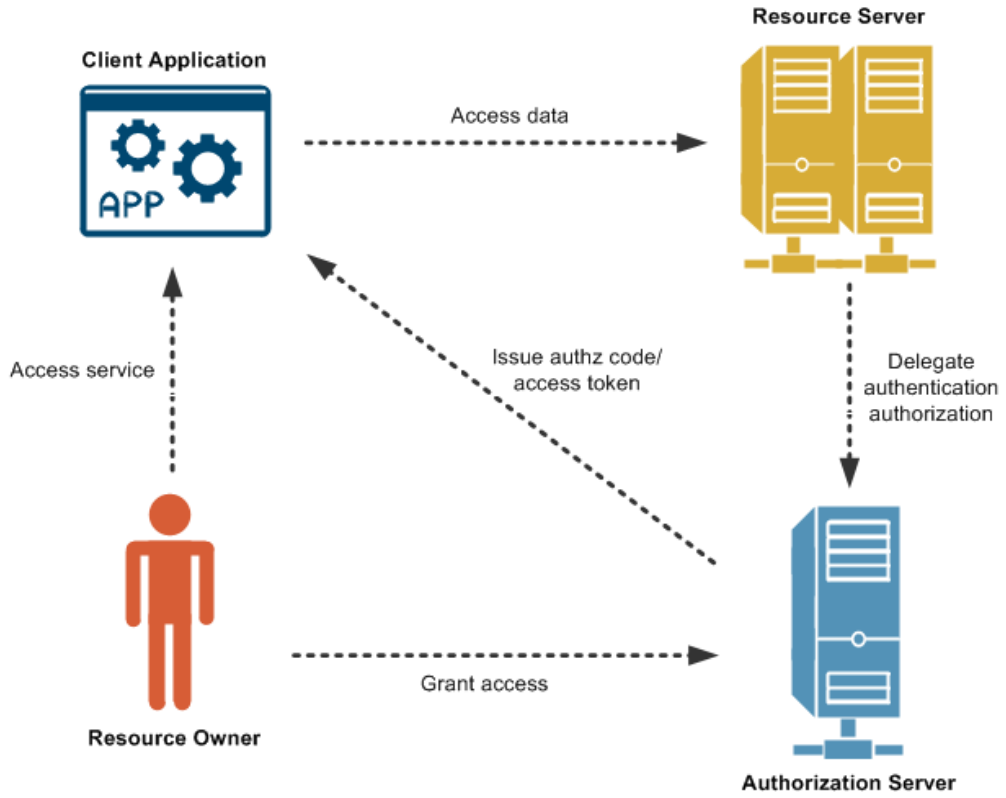


Image Credit: https://docs.oracle.com/cd/E50612_01/doc.11122/oauth_guide/content/oauth_intro.html

OAuth2 Limits

- Does not prevent misuse of correctly issued tokens
  - A token that is stolen while still valid has a perfectly valid signature
  - It will pass every validation check
  - The protocol cannot distinguish a legitimate token holder from a thief who obtained the token in transit

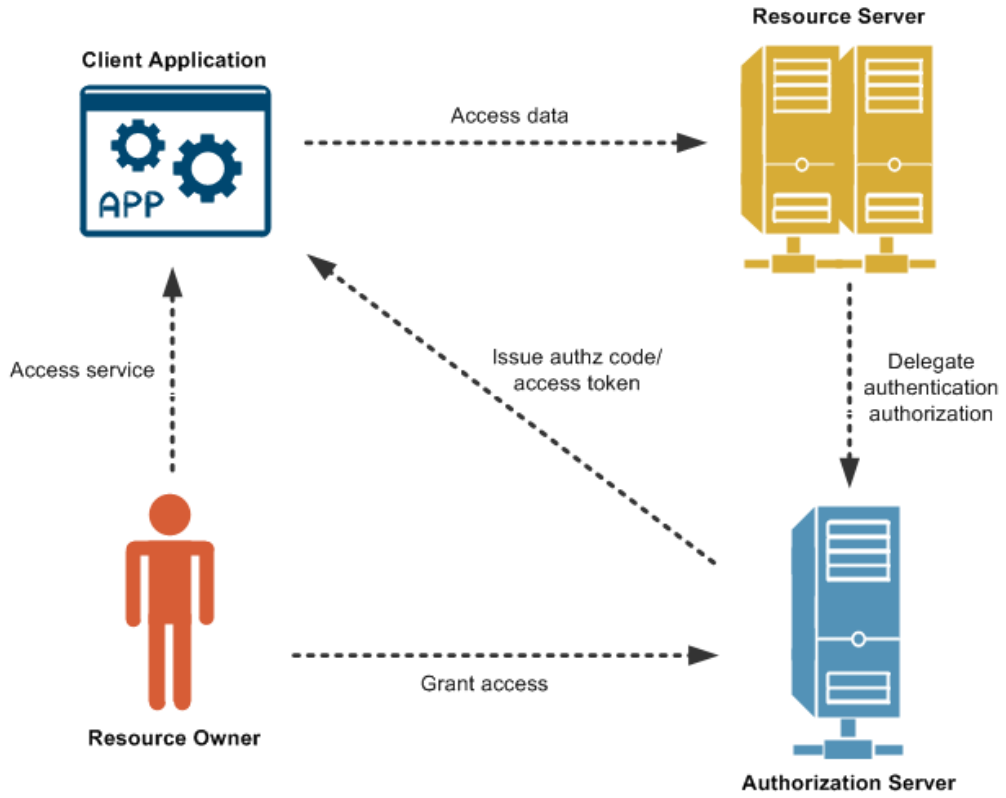


Image Credit: https://docs.oracle.com/cd/E50612_01/doc.11122/oauth_guide/content/oauth_intro.html

OAuth2 Limits

- Secure OAuth2 implementations require layering additional protections
  - PKCE to prevent authorization code interception
  - Short token lifetimes to bound the damage window from token leakage
  - HTTPS on every endpoint involved in token issuance and use
  - Audience validation to prevent tokens issued for one API being replayed against another
  - Resource server scope enforcement to prevent tokens being used for operations they were not granted

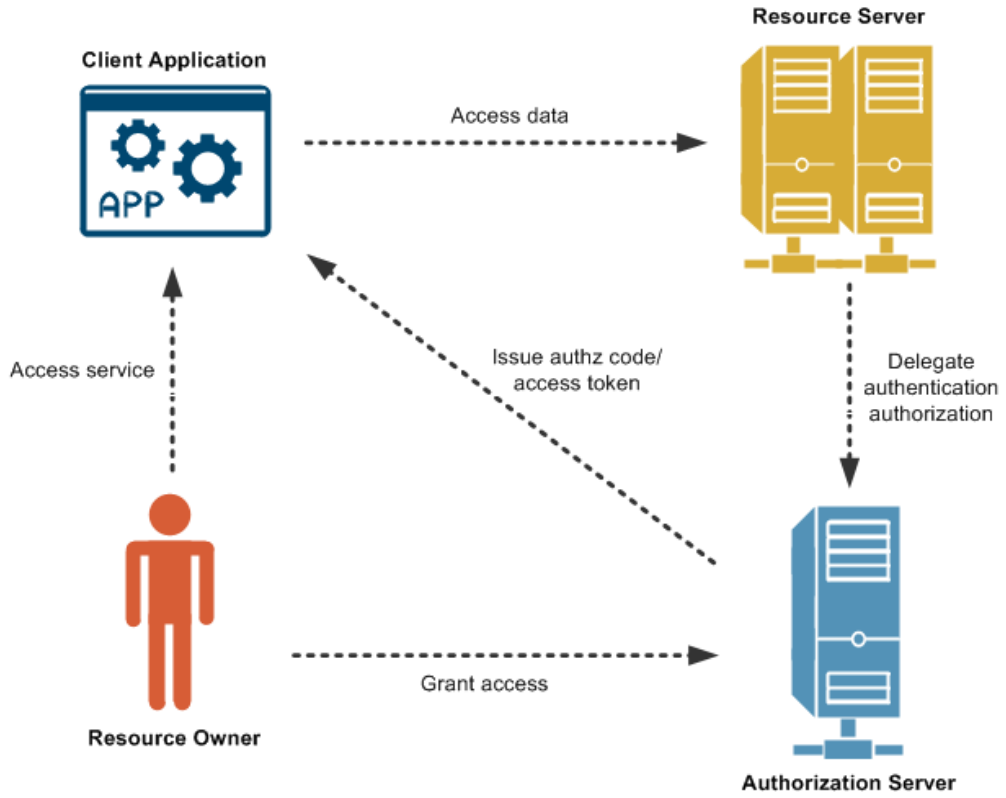


Image Credit: https://docs.oracle.com/cd/E50612_01/doc.11122/oauth_guide/content/oauth_intro.html

The Four Roles

The Roles

- OAuth2 defines a protocol involving four distinct roles
  - Every OAuth2 interaction can be fully described by identifying which participant plays which role
  - Regardless of which grant type is used
  - Regardless of which authorization server is involved

Role	Responsibility in the Protocol
Resource Owner	The entity that owns the protected resources and has the authority to grant access to them. In interactive flows this is a human user who has an account with the authorisation server. In machine-to-machine flows — where no user is present — the resource owner is the system or service itself.
Client	The application that wants access to protected resources on behalf of the resource owner. The client requests tokens from the authorisation server and presents those tokens to the resource server. It never handles the resource owner's credentials.
Authorisation Server (AS)	The trusted authority that authenticates the resource owner, obtains consent where required, and issues tokens. The AS owns the complete token lifecycle: it mints tokens, signs them with its private key, and publishes the public keys needed to verify them. Examples: Keycloak, Auth0, Azure Active Directory, Okta.
Resource Server (RS)	The API that holds the protected resources the client wants to access. The RS validates incoming tokens on every request — verifying the signature, checking expiry and issuer, and enforcing scope requirements — before allowing or denying the operation.

Client and Resource Server Roles

- Spring Boot application can play both the client role and the resource server role
- Resource server
  - Front end interface sends request to GET /api/v1/employees with a Bearer token in the Authorization header
  - Application validates that token before serving the response
  - Checks the signature, expiry, issuer, and required scope
  - In this interaction, Spring Boot is the Resource Server and the front end interface is the client

Role	Spring Security Starter	What It Configures
Resource Server	`spring-boot-starter-oauth2-resource-server`	JWT validation filter chain, JWKS fetching, token claim extraction
Client	`spring-boot-starter-oauth2-client`	Token acquisition, token storage, automatic token attachment to outbound calls

Client and Resource Server Roles

- Client
  - The Spring Boot application receives the request from the front end interface and validates it
  - Then needs to call a downstream HR data service to fulfill it
  - To call that downstream service, Spring Boot must obtain its own access token from the authorization server and attach it to the outbound call
  - In this interaction, Spring Boot is the Client
  - The downstream service is the Resource Server

Role	Spring Security Starter	What It Configures
Resource Server	`spring-boot-starter-oauth2-resource-server`	JWT validation filter chain, JWKS fetching, token claim extraction
Client	`spring-boot-starter-oauth2-client`	Token acquisition, token storage, automatic token attachment to outbound calls

Client and Resource Server Roles

- Client
  - Both starters can be active in the same application simultaneously
  - Knowing which role is active at which point in a request chain is the prerequisite for diagnosing unexpected 401 and 403 responses correctly

Role	Spring Security Starter	What It Configures
Resource Server	`spring-boot-starter-oauth2-resource-server`	JWT validation filter chain, JWKS fetching, token claim extraction
Client	`spring-boot-starter-oauth2-client`	Token acquisition, token storage, automatic token attachment to outbound calls

Authorization Server

- Authorization server (AS) is the most complex component in the OAuth2 architecture
  - Also the one the application is least directly responsible for
  - And whose configuration has the most significant security consequences
- The AS is responsible for the complete token issuance lifecycle.

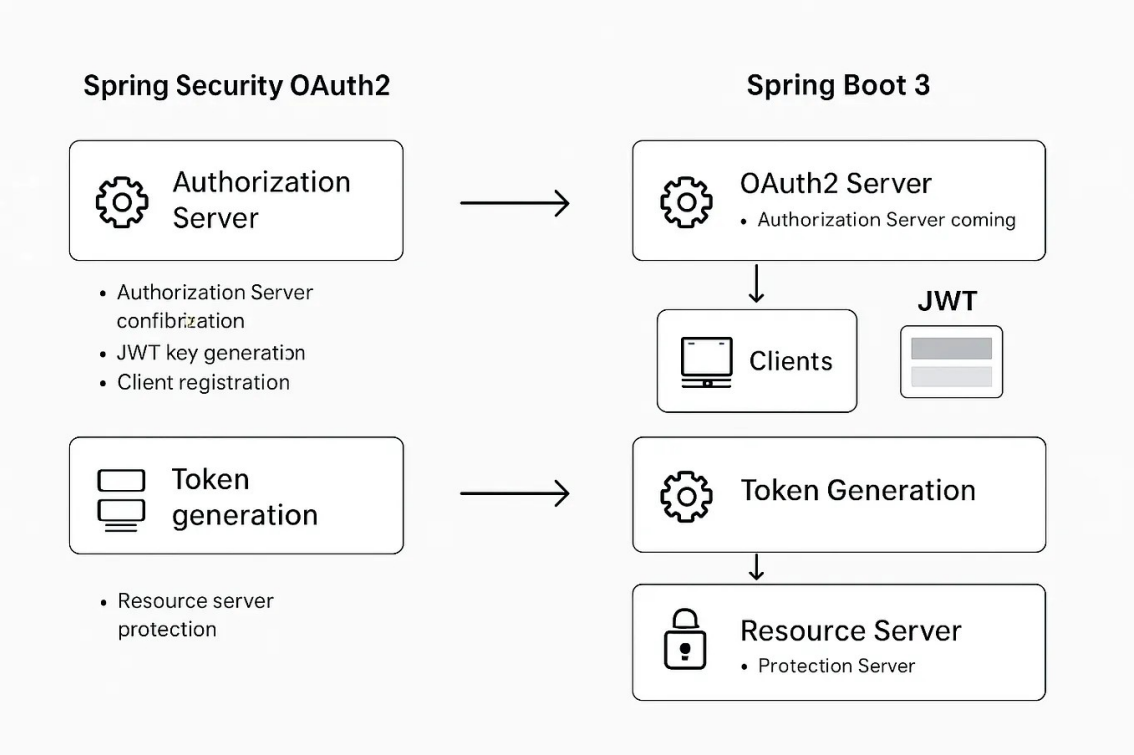


Image Credit: https://medium.com/@dev.jefster/oauth2-in-spring-security-understanding-the-client-authorization-server-and-resource-server-e90c14630b20

Authorization Server

- What the AS guarantees when it issues a token
  - The resource owner was authenticated using the configured authentication method at the time of issuance
  - The resource owner explicitly consented to the requested scopes at the time of issuance
  - The token encodes the agreed scope, the subject identity, the issuer identity, and the expiry time
  - The token was signed with the AS's current private key
  - The signature can be verified by any party with access to the corresponding public key via the JWKS endpoint

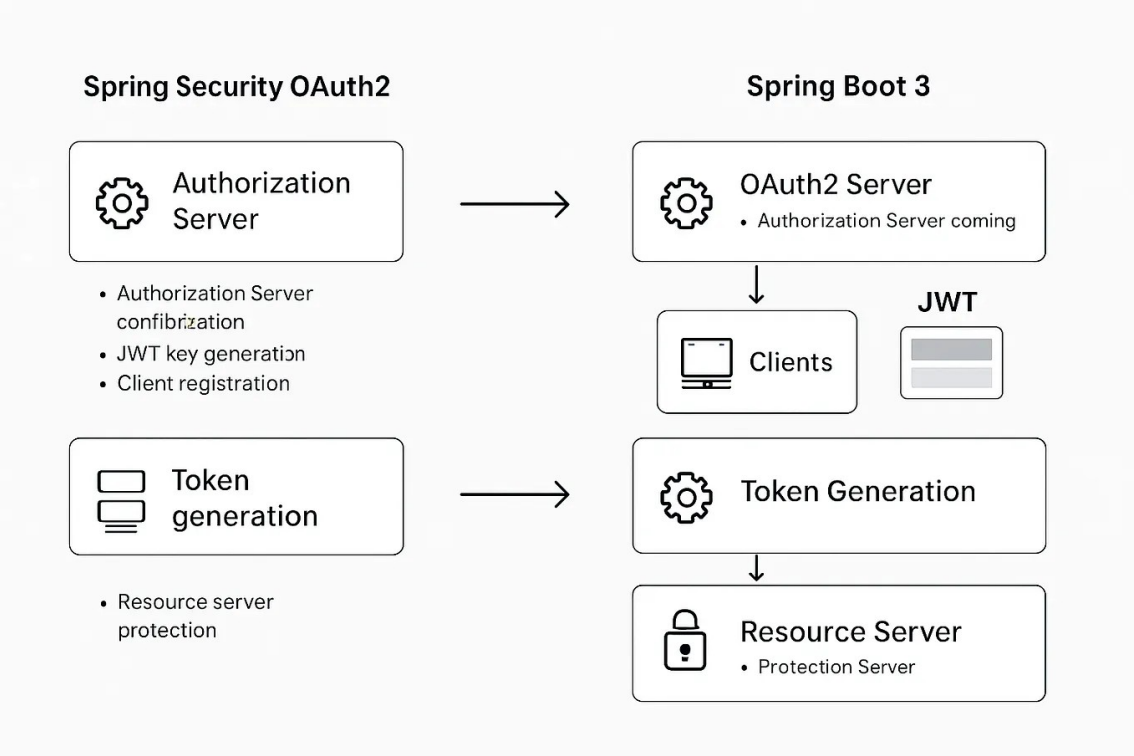


Image Credit: https://medium.com/@dev.jefster/oauth2-in-spring-security-understanding-the-client-authorization-server-and-resource-server-e90c14630b20

Authorization Server

- What the AS does *not* guarantee after issuance
  - That the resource owner still exists or has the same permissions
    - The token reflects the state at issuance, not the current state
  - That the token has not been stolen
    - The AS has no visibility into what happens to the token after it is issued
  - That the resource owner intended each specific API call made using the token
    - Consent was granted to the scope, not to individual operations

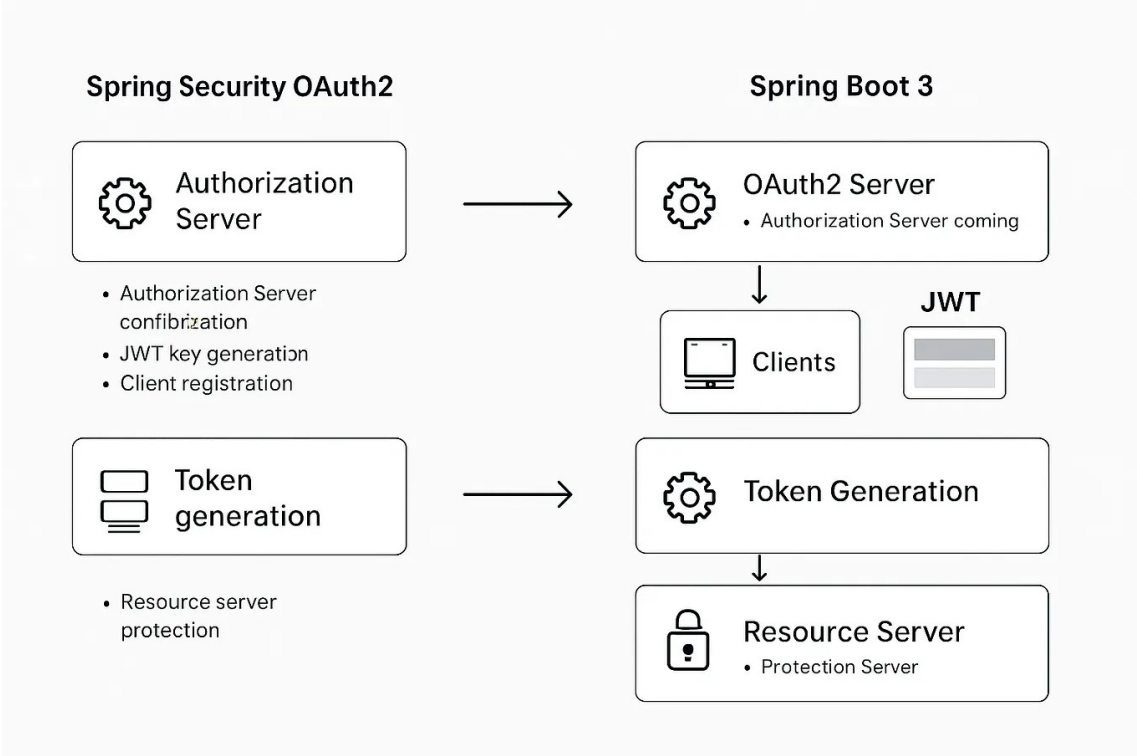


Image Credit: https://medium.com/@dev.jefster/oauth2-in-spring-security-understanding-the-client-authorization-server-and-resource-server-e90c14630b20

Authorization Server

- Practical implication of short token lifetimes
  - Ensure the gap between "state at issuance" and "current state" is bounded
  - A ten-minute access token means the worst case for a revoked permission or a stolen token is ten minute
  - A twenty-four-hour token means a day-long exposure window

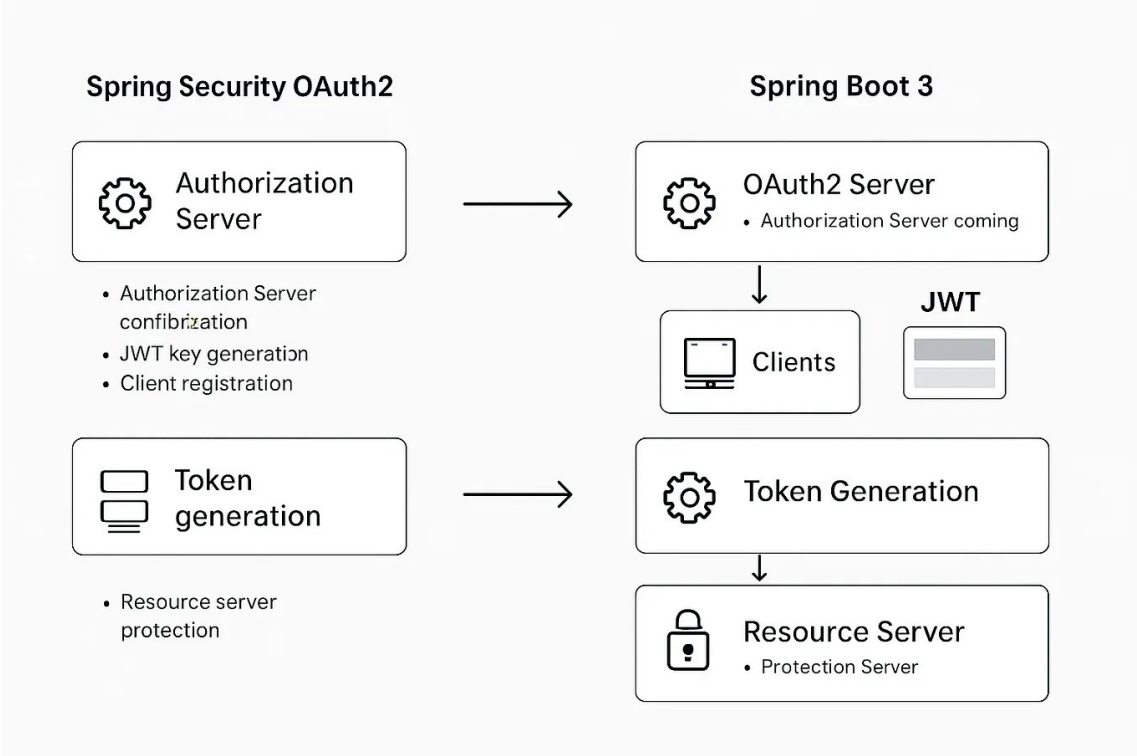


Image Credit: https://medium.com/@dev.jefster/oauth2-in-spring-security-understanding-the-client-authorization-server-and-resource-server-e90c14630b20

The Resource Server

- The resource server
  - Trusts nothing except a cryptographically valid token from the configured authorization server
  - Every request, on every endpoint, is subject to the same validation sequence
  - There are no shortcuts and no exceptions
  - Including for internal callers, service accounts, or requests from trusted networks

Step	What Is Checked	If It Fails
1. Token present	`Authorization: Bearer` header exists and is non-empty	401 — the client must authenticate
2. Signature	Token signature verified against the AS's current public key from the JWKS endpoint	401 — the token was not issued by the trusted AS, or has been tampered with
3. Expiry	Current time is before the `exp` claim	401 — the token has expired, the client must obtain a new one
4. Issuer	The `iss` claim matches the configured authorisation server	401 — the token was issued by a different AS than the one this RS trusts
5. Audience	The `aud` claim includes this resource server's identifier	401 — the token was issued for a different API and cannot be replayed here
6. Scope	The token's scope claims include the permission required by this endpoint	403 — the token is valid but does not grant permission for this operation

The Resource Server

- The resource server
- Steps 1 through 5
  - Is this a valid token from a trusted source?
  - A failure at any of these steps produces 401
  - The client needs to obtain a valid token before proceeding.
- Step 6
  - Does this valid token permit this operation?
  - A failure at step 6 produces 403
  - The token is genuine, but it does not grant the required permission

Step	What Is Checked	If It Fails
1. Token present	`Authorization: Bearer` header exists and is non-empty	401 — the client must authenticate
2. Signature	Token signature verified against the AS's current public key from the JWKS endpoint	401 — the token was not issued by the trusted AS, or has been tampered with
3. Expiry	Current time is before the `exp` claim	401 — the token has expired, the client must obtain a new one
4. Issuer	The `iss` claim matches the configured authorisation server	401 — the token was issued by a different AS than the one this RS trusts
5. Audience	The `aud` claim includes this resource server's identifier	401 — the token was issued for a different API and cannot be replayed here
6. Scope	The token's scope claims include the permission required by this endpoint	403 — the token is valid but does not grant permission for this operation

Grant Types

Determining Grant Types

- A grant type defines the specific protocol exchange used to obtain tokens from the authorization server
- Each grant type is a different conversation between the participants involving
  - A different sequence of HTTP requests
  - A different set of credentials presented
  - A different set of security guarantees provided in return

Grant Type	User Present	Client Type	Status
Authorization Code + PKCE	Yes	Public or confidential	Current standard — use for all interactive flows
Client Credentials	No	Confidential only	Current standard — use for service-to-service
Refresh Token	No (after initial auth)	Public or confidential	Not standalone — used alongside Authorization Code
Device Code	Yes (on a second device)	Public	Limited input devices — not covered in this course
Implicit	Yes	Public	**Deprecated — do not use**
Resource Owner Password	Yes	Confidential	**Deprecated — do not use**

Grant Types

- What type of client is this?
  - A public client cannot securely store a secret
    - A browser-based SPA or a native mobile application for example
  - Anyone who can run the application can extract any secret embedded in it
  - A confidential client can store a secret securely because the runtime environment is not accessible to end users
    - For example, a server-side application running in a controlled environment
  - This distinction determines which credentials the client can legitimately present to the authorization server

Grant Type	User Present	Client Type	Status
Authorization Code + PKCE	Yes	Public or confidential	Current standard — use for all interactive flows
Client Credentials	No	Confidential only	Current standard — use for service-to-service
Refresh Token	No (after initial auth)	Public or confidential	Not standalone — used alongside Authorization Code
Device Code	Yes (on a second device)	Public	Limited input devices — not covered in this course
Implicit	Yes	Public	**Deprecated — do not use**
Resource Owner Password	Yes	Confidential	**Deprecated — do not use**

Grant Types

- What level of trust has the client been granted?
  - Some flows require the client to handle user credentials directly
  - This level of trust should be granted only in specific, controlled circumstances
    - In modern OAuth2 practice, it should not be granted at all
- The grant type is the answer that emerges from these three questions
  - It is determined by the nature of the integration

Grant Type	User Present	Client Type	Status
Authorization Code + PKCE	Yes	Public or confidential	Current standard — use for all interactive flows
Client Credentials	No	Confidential only	Current standard — use for service-to-service
Refresh Token	No (after initial auth)	Public or confidential	Not standalone — used alongside Authorization Code
Device Code	Yes (on a second device)	Public	Limited input devices — not covered in this course
Implicit	Yes	Public	**Deprecated — do not use**
Resource Owner Password	Yes	Confidential	**Deprecated — do not use**

PKCE Protocol

Authorization Code + PKCE

- Authorization Code + PKCE
  - Correct grant type for any flow where a user is present and authenticating interactively
  - Most widely supported
  - Most thoroughly security-reviewed flow in the OAuth2 specification
  - When in doubt about which flow to use, the answer is Authorization Code + PKCE

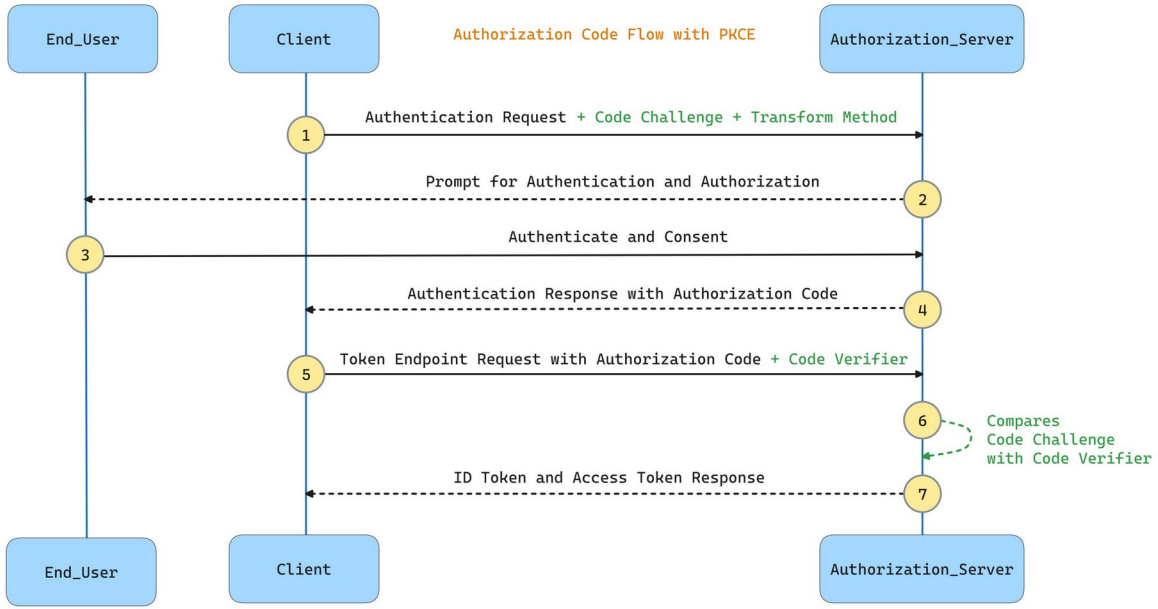


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

Authorization Code + PKCE

- PKCE
  - Proof Key for Code Exchange
  - Security extension defined in RFC 7636
  - Originally designed for public clients that cannot store a client secret
  - Has since become mandatory for all public clients and is strongly recommended for confidential clients
  - It protects against a class of attack that is possible even when a client secret is in use

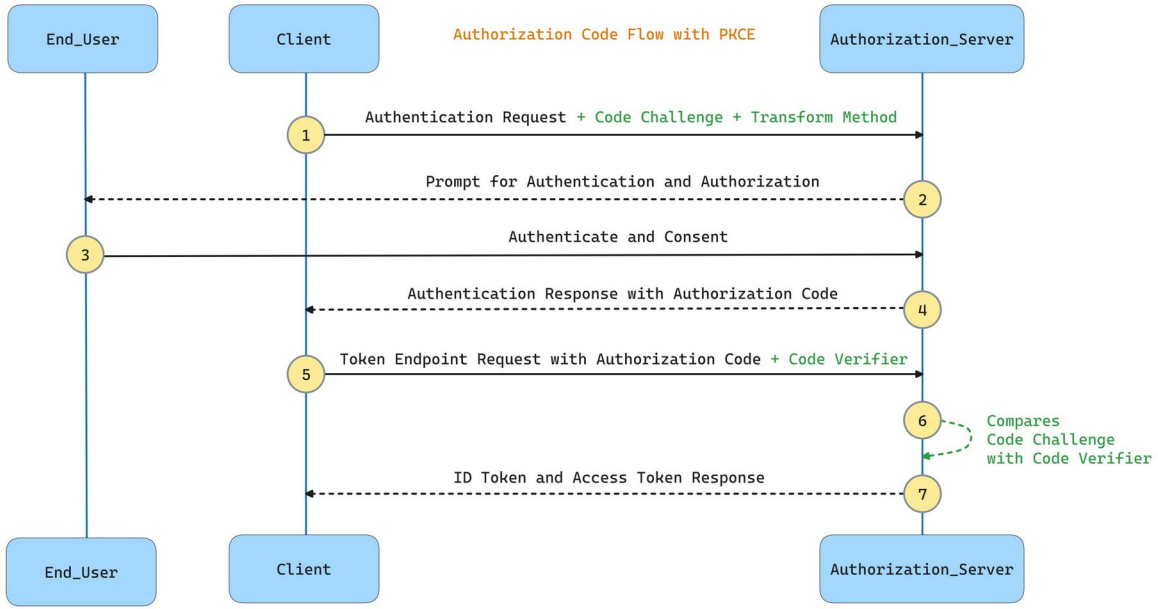


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

Authorization Code + PKCE

- PKCE
  - The attack that PKCE prevents is the authorization code interception attack
  - Without PKCE
    - The authorization server returns a short-lived code to the client via a redirect URL
    - If an attacker can observe or intercept that code they can exchange it for tokens at the token endpoint, because the code alone is sufficient for the exchange
  - Attack executed through
    - URL logging
    - A malicious app registered to the same redirect URI scheme
    - A compromised browser extension

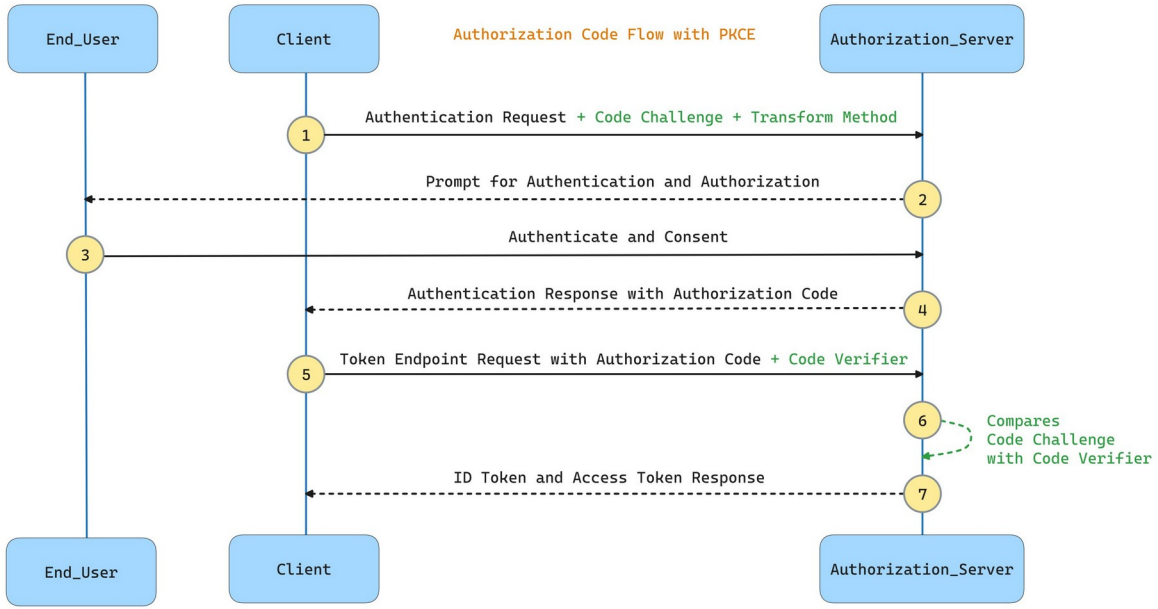


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

Authorization Code + PKCE

- PKCE
  - Closes this gap by cryptographically binding the token exchange to the specific client instance that initiated the authorization request
  - An intercepted token is useless without the corresponding secret that only the originating client holds
- When to use it
  - User interface calling a Spring Boot API
  - Server-side web application with a user login flow
  - Native mobile or desktop application
  - Any flow where a human user is authenticating interactively

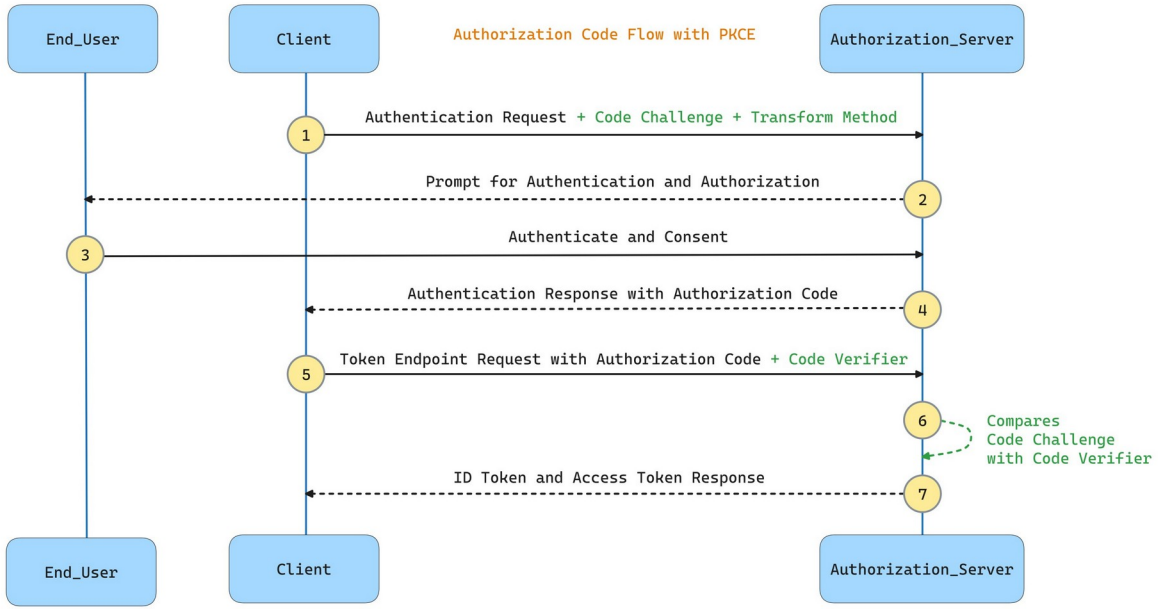


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

Authorization Code + PKCE

- The flow is a nine-step sequence
  - The security properties emerge from the specific design of each step

Step	Actor	Action	Key Security Detail
1	Client	Generate `code_verifier` — a cryptographically random string, 43–128 characters	Must be high-entropy, unique per request. Never reused.
2	Client	Compute `code_challenge` = BASE64URL(SHA256(`code_verifier`))	The challenge is sent to the AS. The verifier stays secret on the client.
3	Browser	Redirect to AS `/authorize` with `client_id`, `response_type=code`, `scope`, `redirect_uri`, `state`, `code_challenge`, `code_challenge_method=S256`	`state` is a per-request CSRF token. `redirect_uri` must be pre-registered with the AS.
4	AS	Authenticate the user — login form, MFA, SSO. Present consent screen if required.	The AS handles all credential processing. The client never sees the user's password.
5	AS → Browser	Redirect to `redirect_uri` with `code` and `state`	`code` is valid for 60–300 seconds, single use. Verify `state` matches before proceeding.

Authorization Code + PKCE

- The state parameter is not optional
  - It is a CSRF protection mechanism
  - Without it, an attacker can construct a malicious authorization URL and trick a user's browser into completing the flow with an attacker-controlled code
  - This would bind the victim's session to the attacker's account
  - The state value must be generated randomly per request, stored before the redirect, and verified on return.
  - A mismatch must terminate the flow immediately

6	Client → AS	POST to `/token` with `grant_type=authorization_code`, `code`, `redirect_uri`, `client_id`, `code_verifier`	AS hashes the `code_verifier` and compares to the stored `code_challenge`. Mismatch = rejection.
7	AS → Client	Return `access_token`, `id_token`, `expires_in`, `refresh_token` (if `offline_access` scope requested)	Both tokens are sensitive credentials. Store securely. Never log their values.
8	Client → RS	Call the resource server with `Authorization: Bearer <access_token>`	Never send the token as a URL query parameter — it appears in logs, history, and referrer headers.
9	RS	Validate token — fetch JWKS, verify signature, check `exp` / `iss` / `aud`, enforce required scopes	Every request. Every time. No exceptions.

THE *AUTHORIZATION CODE* FLOW WITH PKCE

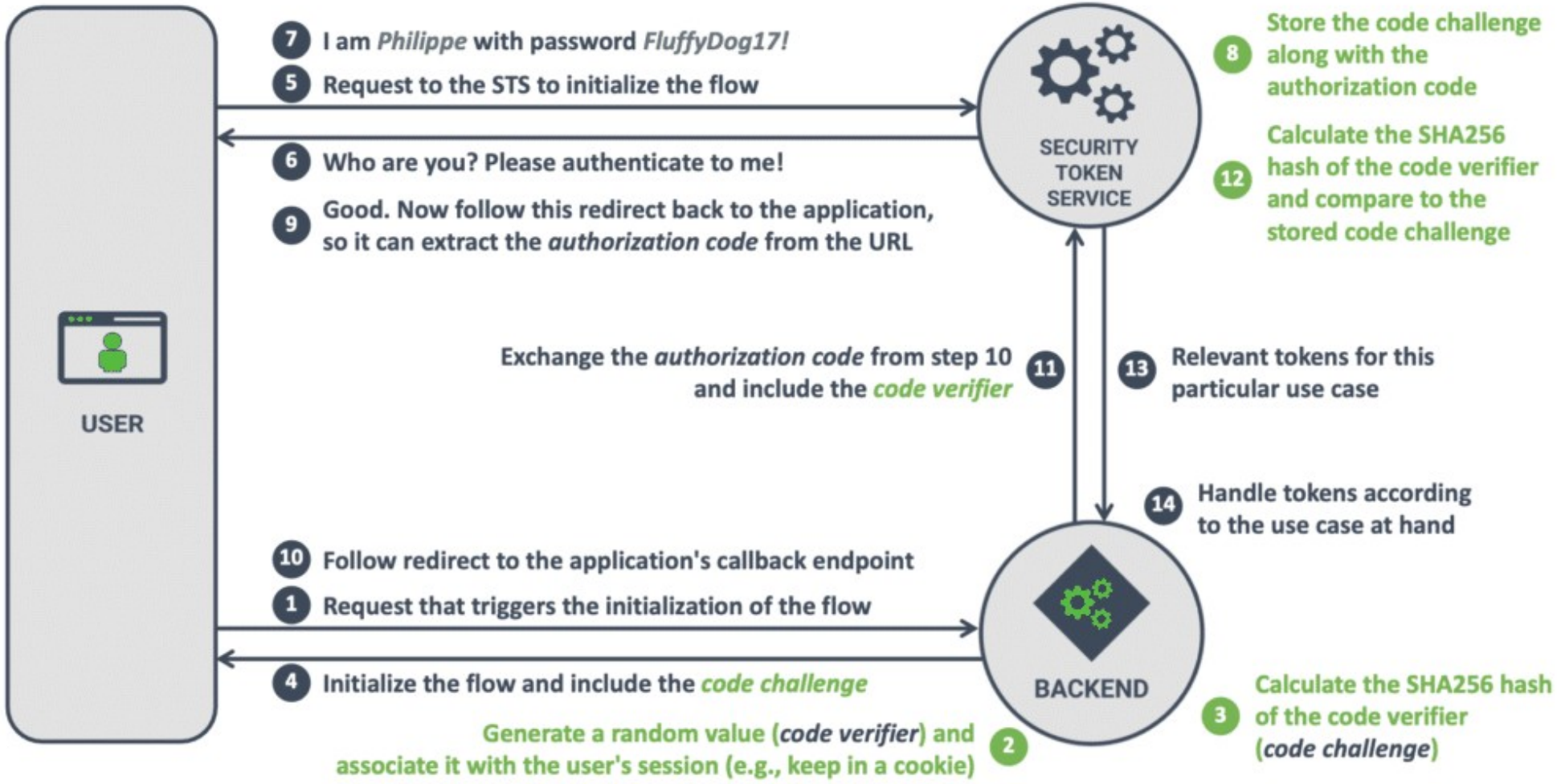


Image Credit: https://www.continuum.be/blog/authentication-authorization-and-pixies-how-does-it-work/

Client Credentials: Service-to-Service

- Client Credentials flow
  - Correct grant type when one service calls another service and no user is present in the interaction
  - The client authenticates using its own identity
    - Its client_id and client_secret
  - Receives an access token representing the client itself, not any user

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=employee-service
&client_secret=<secret>
&scope=read:hr-data write:hr-events

Response:
{
  "access_token": "eyJhbGci...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "read:hr-data write:hr-events"
}
```

Client Credentials: Service-to-Service

- When to use it
  - A Spring Boot service calling a downstream HR data API with no user context
  - A background job that needs to call another service on a schedule
  - Any service-to-service integration where the operation is performed on behalf of the system, not a specific user

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=employee-service
&client_secret=<secret>
&scope=read:hr-data write:hr-events

Response:
{
  "access_token": "eyJhbGci...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "read:hr-data write:hr-events"
}
```

Client Credentials: Service-to-Service

- The fundamental difference
- In authorization code + PKCE
  - The token represents a user's delegated authorization
  - The sub claim in the token is a user identifier
- In client credentials
  - The token represents the service's own identity
  - The sub claim is the client identifier
  - There is no user, no consent screen, and no redirect.
  - The exchange is a direct, synchronous credential-for-token call

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id=employee-service
&client_secret=<secret>
&scope=read:hr-data write:hr-events

Response:
{
  "access_token": "eyJhbGci...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "read:hr-data write:hr-events"
}
```

Refresh Tokens

- The refresh token grant is not a standalone flow
  - Used alongside authorization code + PKCE
  - Allows a client to obtain a new access token after the current one expires, without requiring the user to log in again
- Access tokens are intentionally short-lived
  - Typically 5 to 15 minutes in high-security environments, up to one hour for lower-risk APIs
  - Short lifetimes limit the damage window if a token is stolen
  - Terrible user experience if the user has to re-authenticate every 15 minutes

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token=<refresh_token_value>
&client_id=my-spa
```

Refresh Tokens

- The refresh token bridges this gap
  - Longer-lived credential
  - Issued alongside the access token when the offline_access scope is requested
  - Can be exchanged for a new access token at any time without user interaction
- A refresh token must be rotated on every use.
  - When the client exchanges a refresh token
    - The authorization server issues a new refresh token alongside the new access token, and invalidates the old one
  - If an attacker attempts to the token after it has been rotated it
    - AS detects the reuse of an already-invalidated token and can invalidate the entire token family
    - Cutting off both the attacker and prompting the legitimate user to re-authenticate

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token=<refresh_token_value>
&client_id=my-spa
```

Refresh Tokens

- Refresh tokens are long-lived and sensitive
  - A stolen refresh token provides extended access, bounded only by its own expiry
  - In a browsers, refresh tokens should be stored in an httpOnly, Secure, SameSite=Strict cookie
  - Not stored in localStorage
    - Or it is accessible to JavaScript and therefore vulnerable to cross-site scripting

```
Set-Cookie: refresh_token=eyJhbGci...;
            HttpOnly;          ← JavaScript cannot read this
            Secure;            ← Only sent over HTTPS, never HTTP
            SameSite=Strict;   ← Only sent with same-site requests
            Path=/auth;        ← Only sent to the token refresh endpoint
            Max-Age=604800     ← Expires in 7 days
```

Deprecated Flows

- The Implicit flow and the Resource Owner Password Credentials flow are both deprecated
  - They remain in the OAuth2 specification for historical completeness
- Implicit flow
  - Designed for browser-based public clients that could not use the Authorization Code flow because they could not keep a client secret
  - Returned the access token directly in the URL fragment after the redirect

```
https://app.example.com/callback
  #access_token=eyJhbGci...    ← token in URL fragment
  &token_type=Bearer
  &expires_in=3600
```

Deprecated Flows

- Implicit flow
  - The token was returned in one of the least secure parts of the HTTP exchange
  - There was no mechanism to verify that the token delivered to the page was intended for that specific client instance
  - PKCE solved the problem the Implicit flow was attempting to solve
  - Any application that uses the Implicit flow should be migrated to Authorization Code + PKCE

```
https://app.example.com/callback
  #access_token=eyJhbGci...    ← token in URL fragment
  &token_type=Bearer
  &expires_in=3600
```

Deprecated Flows

- Resource Owner Password Credentials flow
  - Allowed the client application to collect the user's username and password directly
  - Then exchange them for tokens at the AS token endpoint
  - The application handled the user's credential
  - Eliminates the security boundary between the user and the client
    - If the client is compromised, the attacker has the password
  - Any system still using the ROPC flow should be treated as a security vulnerability

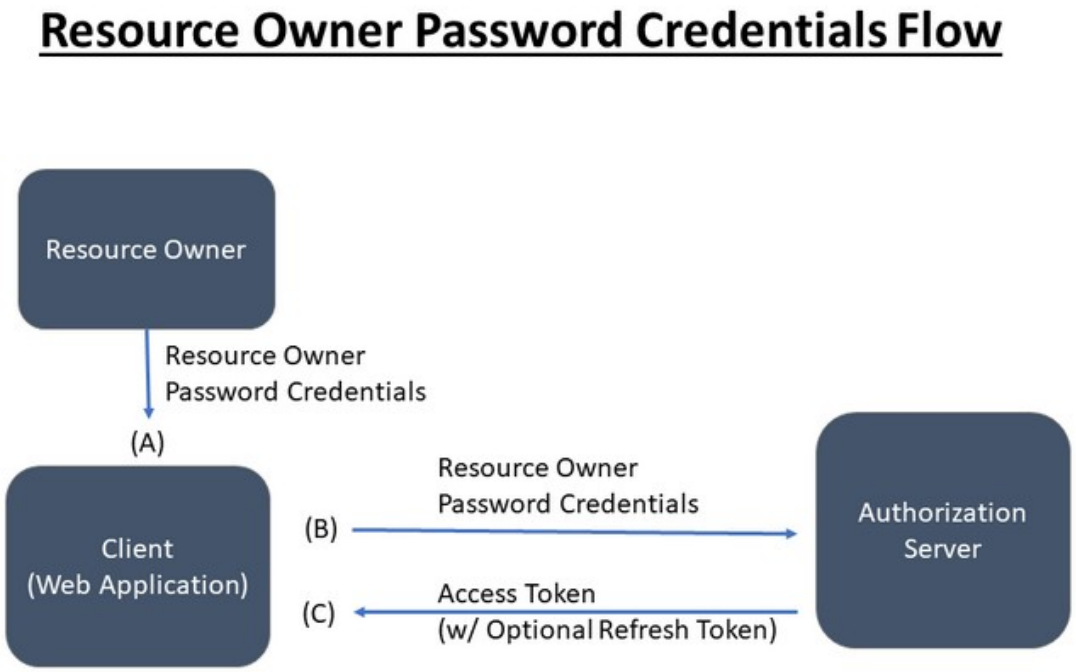


Image Credit: https://secumantra.com/oauth2-flows-part-2/

Choosing the Right Flow

- Is a user present in the interaction?
- If yes
  - Use Authorization Code + PKCE
  - No exceptions for interactive flows
- If no
  - Use Client Credentials
  - The calling service authenticates with its own identity
  - No user, no redirect, no consent screen

Scenario	Grant Type	Reason
React SPA → Spring Boot API	Authorization Code + PKCE	User is present. SPA is a public client. PKCE is mandatory.
Spring Boot API receiving SPA requests	Resource Server validation	Not a grant type — validates the token the SPA obtained
Spring Boot API → downstream service	Client Credentials	No user present. Service authenticates with its own identity.
Background job → internal API	Client Credentials	Automated, no user. Service-to-service.
Keeping a user session alive	Refresh Token (alongside Auth Code)	Not standalone — renews the access token silently after expiry
Legacy system using Implicit flow	**Migrate to Authorization Code + PKCE**	Implicit is deprecated. No new implementations.
Legacy system using ROPC	**Treat as a security finding**	ROPC is deprecated. Client handles user credentials directly.

JWT Tokens

JSON Web Token

- A JWT is a three-part structure
  - Header . Payload . Signature
    - Each part Base64URL-encoded
    - Separated by dots
  - Designed to be readable
    - header and payload are plain JSON, decodable by anyone
  - Designed to be tamper-evident
    - The signature is cryptographically bound to the content
    - If any part of the header or payload changes after signing, the signature fail
  - The token is not encrypted by default
    - Never put sensitive data in the payload

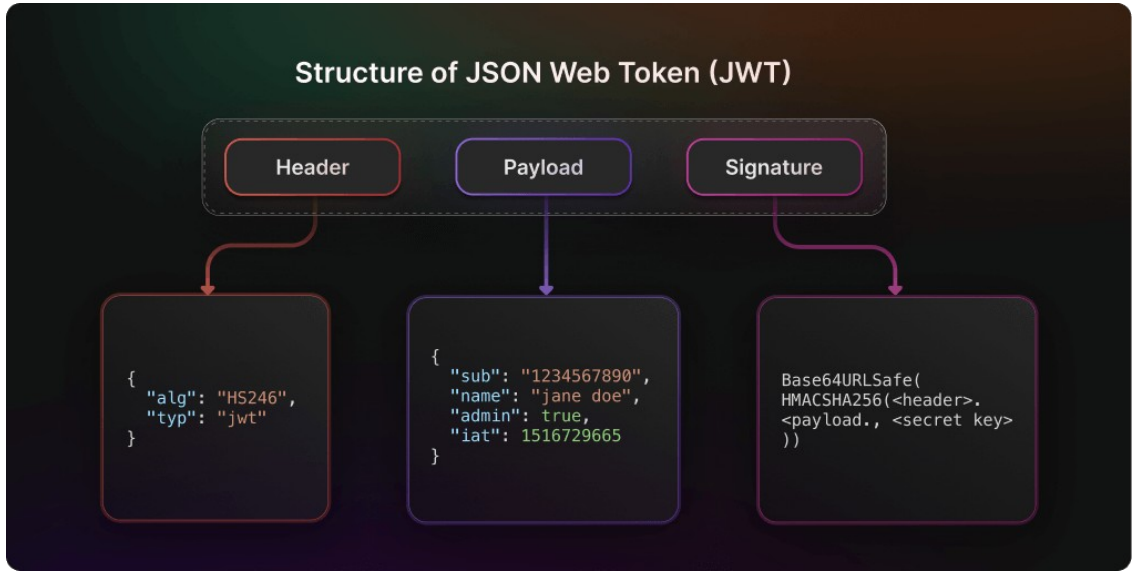


Image Credit: https://supertokens.com/jwt-encoder-decoder

JSON Web Token

- A JWT is a three-part structure
- Design principles
  - Any JWT can be pasted into jwt.io and read the header and payload immediately
  - Useful for debugging
  - But it also means the token is not a secret itself
  - The secret is the signature key
    - Only the authorization server holds it

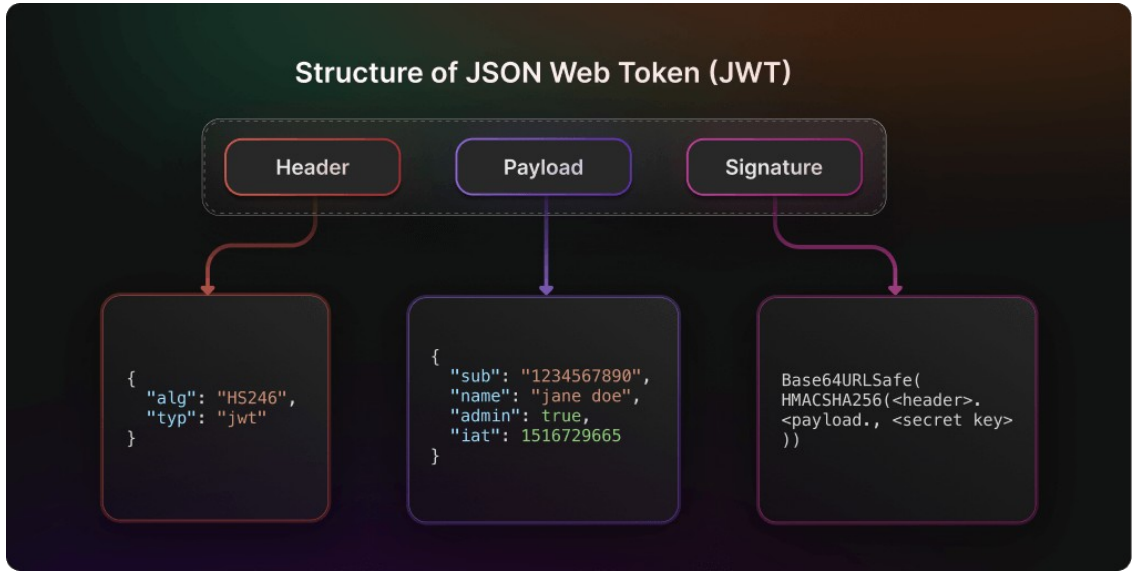


Image Credit: https://supertokens.com/jwt-encoder-decoder

JSON Web Token

- The Header
  - The header tells the resource server how to verify the token
  - alg: the signing algorithm
    - The RS uses this to know which verification algorithm to apply
  - typ: the token type
    - We are only concerned with JWT tokens
  - kid: Key ID
    - Tells the resource server which specific key in the JWKS endpoint to use for verification
    - Configuration of the endpoint is shown on the right

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "key-identifier-123"
}
```

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwks-uri: https://auth.example.com/realms/dev/protocol/openid-connect/cert
```

JSON Web Token

- The Header
  - RS256 algorithm
  - RS256 uses a private/public key pair
  - The AS signs with the private key
  - Any RS verifies with the public key
  - Prefer RS256 over HS256 (symmetric) for APIs
  - With HS256 every resource server must share the secret, which is a significant key management problem

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "key-identifier-123"
}
```

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwks-uri: https://auth.example.com/realms/dev/protocol/openid-connect/cert
```

JSON Web Token

- The Payload
  - The payload carries claims
  - Claims are statements about the subject and the token itself
  - Registered claims are defined by RFC 7519 and have reserved names

Claim	Meaning	Why the RS checks it
`sub`	Subject — unique identifier of the user or service	Identifies who the token was issued for. Used in audit logs.
`iss`	Issuer — the authorisation server's URL	Must match the configured issuer URI. Prevents tokens from a different AS being accepted.
`aud`	Audience — which resource server this token is for	Must include this RS's identifier. Prevents token replay across APIs.
`exp`	Expiry — Unix timestamp after which the token is invalid	RS rejects any token past this value. Core time-bounding control.
`iat`	Issued At — Unix timestamp of issuance	Used to calculate token age. Supports clock skew detection.
`jti`	JWT ID — unique identifier for this specific token	Enables token revocation checking and replay detection.

JSON Web Token

- The Payload
  - The AS adds custom claims based on the user's profile and the client's configuration.
  - scope: the permissions granted to the client application
    - Defined by the resource server, requested by the client, approved by the user
  - roles: the organizational roles of the authenticated user
    - Custom claim added by the AS from the user's profile in the identity provider
    - Not a registered claim so name and structure vary by AS configuration

```
{
  "sub":   "a3f8b2c1-7d4e-4a9f-b831-2e5c9d0f1a72",
  "iss":   "https://auth.example.com/realms/hr-platform",
  "aud":   ["employee-api", "reporting-api"],
  "exp":   1735689600,
  "iat":   1735686000,
  "jti":   "9c1d3a88-22e9-4f1b-7d0c-5e2f91a83b47",

  "scope": "read:employees write:employees read:departments openid profile email",

  "roles":        ["HR_MANAGER", "DEPARTMENT_VIEWER"],
  "department":   "Human Resources",
  "employee_id":  "EMP-00847",

  "name":               "Jane Smith",
  "given_name":         "Jane",
  "family_name":        "Smith",
  "email":              "jane.smith@example.com",
  "email_verified":     true,
  "preferred_username": "jane.smith"
}
```

JSON Web Token

- The Payload
  - name, email: OIDC standard claims
    - Carry verified identity information about the authenticated user
    - Present when the profile and email scopes were requested.
- RFC 7519 JWT specification
  - Specifies three types of claims
- Registered claims
  - Names defined by the spec
  - These names are reserved
  - Cannot be used for a different purpose than their defined meaning

```
{
  "sub":   "a3f8b2c1-7d4e-4a9f-b831-2e5c9d0f1a72",
  "iss":   "https://auth.example.com/realms/hr-platform",
  "aud":   ["employee-api", "reporting-api"],
  "exp":   1735689600,
  "iat":   1735686000,
  "jti":   "9c1d3a88-22e9-4f1b-7d0c-5e2f91a83b47",

  "scope": "read:employees write:employees read:departments openid profile email",

  "roles":        ["HR_MANAGER", "DEPARTMENT_VIEWER"],
  "department":   "Human Resources",
  "employee_id":  "EMP-00847",

  "name":               "Jane Smith",
  "given_name":         "Jane",
  "family_name":        "Smith",
  "email":              "jane.smith@example.com",
  "email_verified":     true,
  "preferred_username": "jane.smith"
}
```

JSON Web Token

- Public claims
  - Names intended for use across organizations and systems
  - The spec says these must be registered in the IANA JSON Web Token Claims Registry to avoid collisions
- Private claims
  - Custom names agreed between the AS and the resource servers within a system
  - Only one practical constraint
  - They should not collide with registered or public claim names

```
{
  "sub":   "a3f8b2c1-7d4e-4a9f-b831-2e5c9d0f1a72",
  "iss":   "https://auth.example.com/realms/hr-platform",
  "aud":   ["employee-api", "reporting-api"],
  "exp":   1735689600,
  "iat":   1735686000,
  "jti":   "9c1d3a88-22e9-4f1b-7d0c-5e2f91a83b47",

  "scope": "read:employees write:employees read:departments openid profile email",

  "roles":        ["HR_MANAGER", "DEPARTMENT_VIEWER"],
  "department":   "Human Resources",
  "employee_id":  "EMP-00847",

  "name":               "Jane Smith",
  "given_name":         "Jane",
  "family_name":        "Smith",
  "email":              "jane.smith@example.com",
  "email_verified":     true,
  "preferred_username": "jane.smith"
}
```

JSON Web Token

- The Signature
  - Cryptographic hash of the encoded header and payload, signed with the AS's private key
- How the RS verifies it
  - Fetches the AS's public keys from the JWKS endpoint
  - Uses the kid in the header to select the correct key
  - Recomputes the hash from the received header and payload
  - Compares against the signature
  - Any mismatch means rejection

```
signature = RSA_SHA256(
  base64url(header) + "." + base64url(payload),
  AS_private_key
)
```

JSON Web Token

- The Signature
- A valid signature proves
  - The token was issued by the authorization server that holds the private key
  - The header and payload have not been modified since issuance
- A valid signature does NOT prove
  - The token has not been stolen
  - The user still has the permissions in the token
  - Roles and scopes reflect the state at issuance, not the current state

```
signature = RSA_SHA256(
  base64url(header) + "." + base64url(payload),
  AS_private_key
)
```

JSON Web Token

- Signature Guarantee Gap
  - The gap between what the signature proves and what operators assume it proves
- Stolen token problem
  - A token is a bearer credential, whoever holds a valid, unexpired token can use it
  - If a token is intercepted in transit, extracted from insecure storage, or obtained via an XSS attack, it will pass every signature validation check
  - The RS has no way to distinguish a legitimate holder from a thief

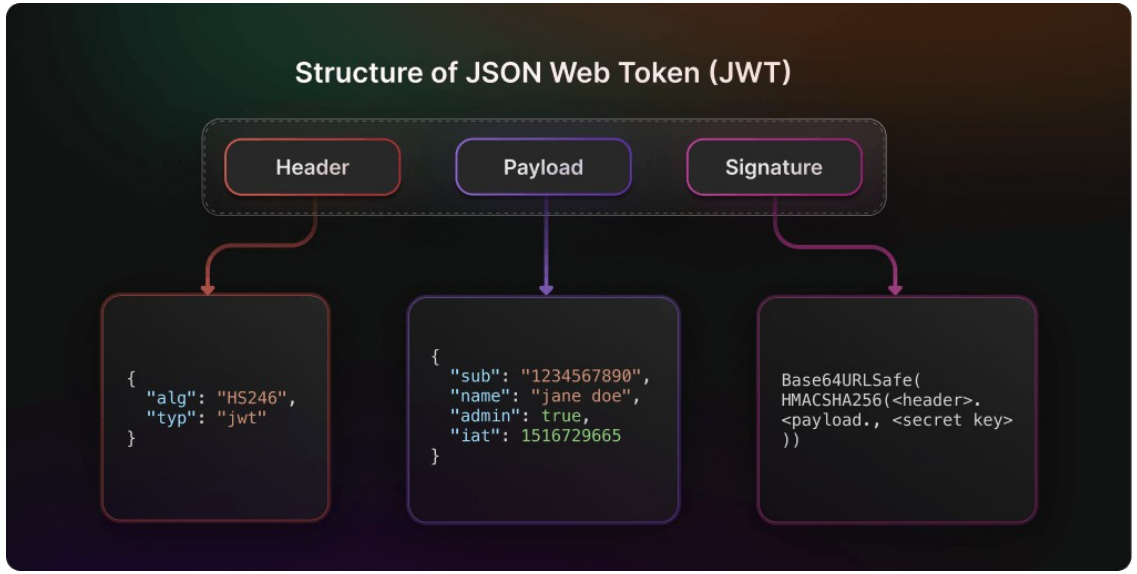


Image Credit: https://supertokens.com/jwt-encoder-decoder

JSON Web Token

- Signature Guarantee Gap
  - The gap between what the signature proves and what operators assume it proves
- Stale permissions problem
  - A user is assigned a role at 09:00 and authenticates and the role appears in the token
  - At 10:00 the user's role is revoked in the identity provider
  - At 10:30 the user makes a request with the token issued at 09:00, the RS validates the signature and grants access
    - The token accurately reflects the state at issuance, not the current state

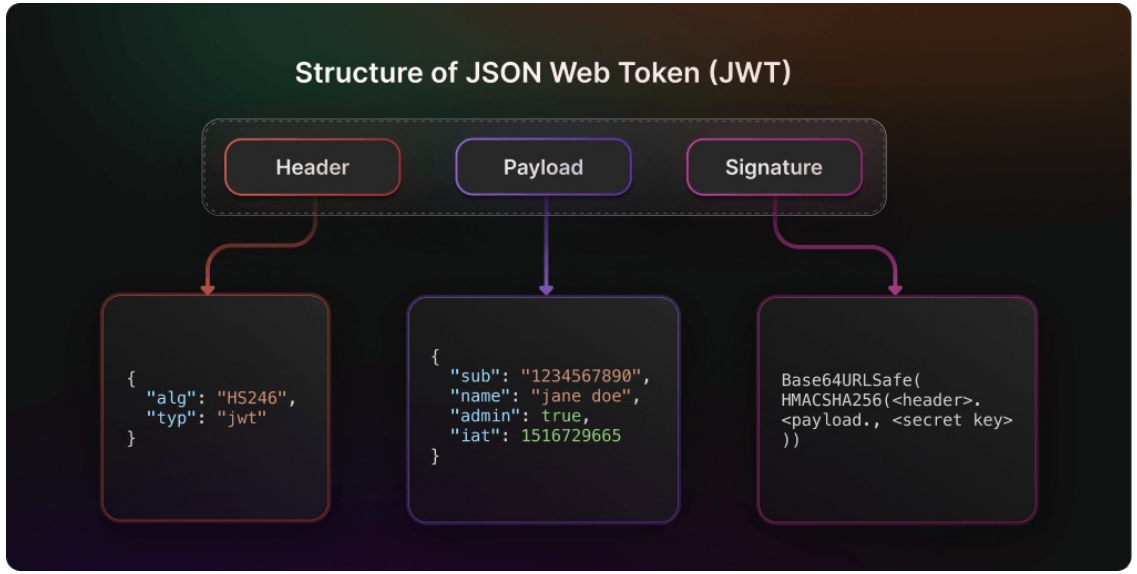


Image Credit: https://supertokens.com/jwt-encoder-decoder

JSON Web Token

- Signature Guarantee Gap
  - The gap between what the signature proves and what operators assume it proves
- Stale permissions problem
  - A user is assigned a role at 09:00 and authenticates and the role appears in the token
  - At 10:00 the user's role is revoked in the identity provider
  - At 10:30 the user makes a request with the token issued at 09:00, the RS validates the signature and grants access
    - The token accurately reflects the state at issuance, not the current state

Problem	Mitigation	Trade-off
Stolen token	Short access token lifetime (5–15 min)	More frequent silent refreshes
Stale permissions	Short lifetime + re-authentication on sensitive operations	User friction on high-risk actions
High-security revocation	Token introspection — RS calls AS on every request to verify current validity	AS becomes a synchronous dependency; latency cost

JSON Web Token

- Scope check (API-level access):
  - Does this token have the permission to call this endpoint at all?
  - A token without read:employees scope cannot call GET /api/v1/employees, regardless of who the user is
  - This is a coarse-grained gate enforced at the endpoint level
  - An API gateway can enforce scope rules without any business logic knowledge

```
// Scope check — enforced in SecurityFilterChain
.requestMatchers(HttpMethod.GET, "/api/v1/employees/**")
    .hasAuthority("SCOPE_read:employees")

// Role check — enforced in service or method security
@PreAuthorize("hasRole('HR_MANAGER')")
public EmployeeDto updateSalary(Long id, SalaryUpdateRequest req) { ... }
```

JSON Web Token

- Role check (business logic access):
  - Does this authenticated user have the organizational authority to perform this operation?
  - A token with write:employees scope can reach the salary update endpoint
  - But only a user with the HR_MANAGER role can actually update a salary
  - This is a fine-grained gate enforced inside the service logic

```
// Scope check – enforced in SecurityFilterChain
.requestMatchers(HttpMethod.GET, "/api/v1/employees/**")
    .hasAuthority("SCOPE_read:employees")

// Role check – enforced in service or method security
@PreAuthorize("hasRole('HR_MANAGER')")
public EmployeeDto updateSalary(Long id, SalaryUpdateRequest req) { ... }
```

JSON Web Token

- Mixing scope and role semantics into a single claim
  - Common shortcut that creates specific, predictable security problems
  - Impossible to audit separately
    - Cannot answer "which clients have write access to employees?" and "which users have HR Manager authority?" independently
    - Because they are stored in the same field
  - Scope creep in consent
    - Users consent to scopes, not roles
    - If roles appear in the scope claim, users are consenting to grant their organizational authority to the client application, which is semantically wrong

```
// Anti-pattern — a single claim trying to do two jobs

"permissions": ["employees:read", "employees:write", "HR_MANAGER", "PAYROLL_ADMIN"]
```

JSON Web Token

- Mixing scope and role semantics into a single claim
  - Privilege escalation risk
    - if the same claim drives both endpoint access and business logic decisions, a misconfigured scope grant can accidentally confer organizational authority, or vice versa
  - Gateway enforcement breaks
    - An API gateway can enforce scope rules by inspecting the scope claim
    - If roles appear in the scope claim, the gateway's rules become entangled with business logic it should not know about

```
// Anti-pattern — a single claim trying to do two jobs

"permissions": ["employees:read", "employees:write", "HR_MANAGER", "PAYROLL_ADMIN"]
```

JSON Web Token

Concept	What It Models	Who Defines It	Example
Scope	What the client application is permitted to do — the grant granted by the resource owner to the client.	Defined by the resource server. Requested by the client at authorisation time. Approved by the user during consent.	read:employees, write:departments, openid, profile
Role	What the user is permitted to do — the organisational or functional role of the authenticated person.	Defined by the application's business logic. Assigned to users in the identity provider's user management.	HR_MANAGER, PAYROLL_ADMIN, READ_ONLY_AUDITOR

PKCE Flow

- Recall that without PKCE
  - The AS returns a short-lived authorization code to the client via a redirect URL
  - The code alone is sufficient to exchange for tokens at the token endpoint
  - If an attacker intercepts the code they can exchange it for tokens
  - The legitimate client and the attacker both hold the same code and the AS cannot tell them apart

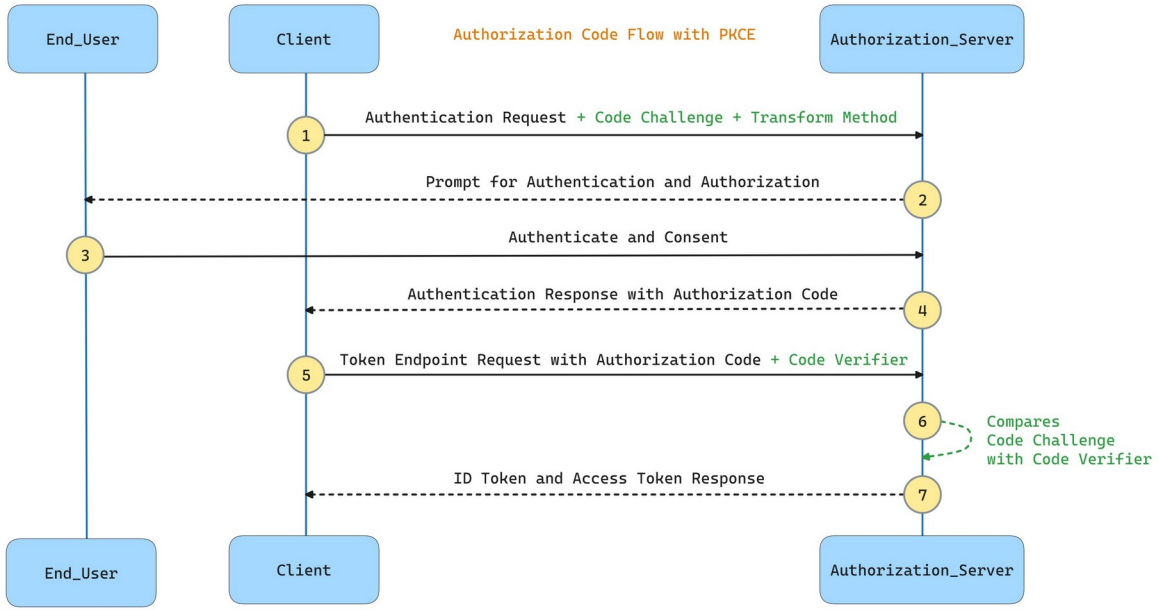


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

PKCE Flow

- What PKCE adds
  - The client generates a secret before the flow starts, the code_verifier
  - Only a hash of that secret (the code_challenge) is sent to the AS
  - When exchanging the code for tokens, the client must prove it holds the original secret
  - An intercepted code without the code_verifier is useless, the exchange will fail

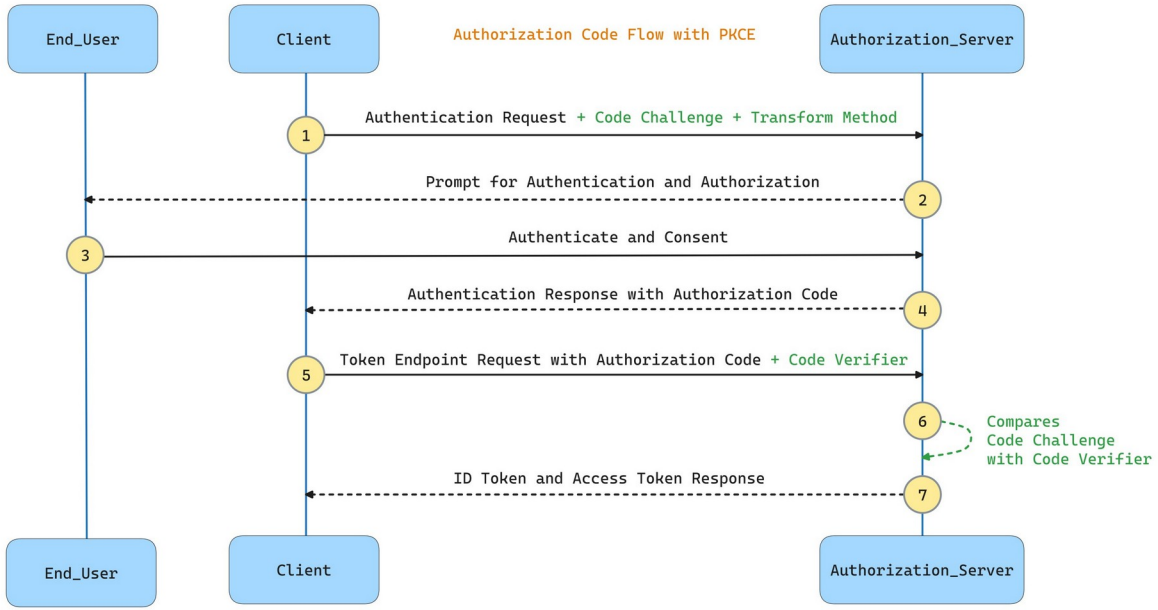


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

PKCE Flow

- Origin and current status
  - Defined in RFC 7636, originally designed for mobile and native public clients
  - Now mandatory for all public clients (SPAs, native apps)
  - Strongly recommended for confidential clients
  - When implementing Authorization Code flow today, PKCE is not optional

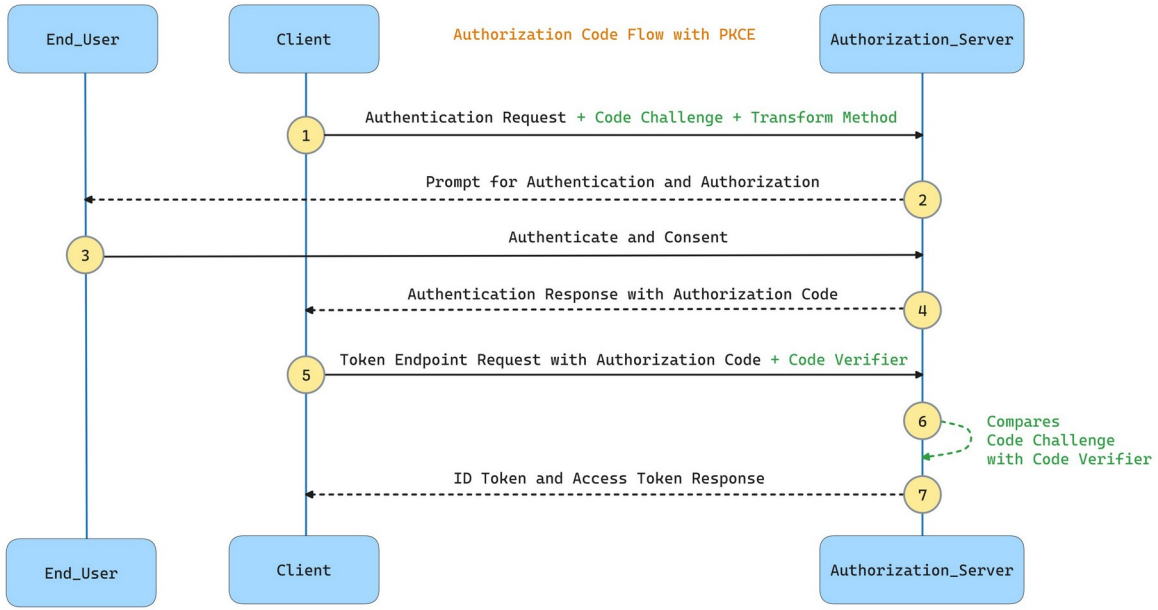


Image Credit: https://auth0.com/blog/pkce-in-web-applications-with-spring-security/

PKCE Flow

- Four participants
  - The security of the flow rests on one property
  - The code_verifier is known only to the client instance that generated it
  - The browser never sees the code_verifier
  - The AS never receives it until the token exchange
  - An attacker who intercepts the redirect URL gets the code
  - But without the code_verifier, the token exchange fails.

Participant	What it holds	What it must never expose
Client	`code_verifier` — the original random secret generated before the flow	The `code_verifier` must never be sent to the browser URL or logged
Authorisation Server	`code_challenge` — the hash of the verifier, stored against the code	The AS's private signing key
Browser	The authorisation `code` returned in the redirect	The code must be exchanged immediately and not stored
Resource Server	The AS's public JWKS keys for token verification	Nothing — the RS only validates, it issues nothing

PKCE Flow

- Step 1: Generate the code_verifier
  - A cryptographically random string, 43–128 characters long
  - URL-safe Base64 characters only: [A-Z], [a-z], [0-9], -, ., _, ~
  - Must be generated fresh for every authorization request and is never reused
  - Must be stored securely for the duration of the flow (in memory, not localStorage)

```
code_verifier = "dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk"
```

PKCE Flow

- Step 2: Compute the code_challenge
  - Apply SHA-256 to the code_verifier
  - Base64URL-encode the result
  - This is what gets sent to the AS
  - Never the verifier itself

```
code_challenge = BASE64URL(SHA256(code_verifier))
               = "E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM"
```

PKCE Flow

- Step 3: The authorization request
  - The browser redirects to the AS /authorize endpoint with all parameters needed for the AS to process the request
  - response_type=code
    - Requests an authorization code, not a token directly
  - scope
    - The permissions being requested
    - openid is required for OIDC
    - Additional scopes are what the client needs

```
https://auth.example.com/realms/hr-platform/protocol/openid-connect/auth
  ?client_id=hr-spa
  &response_type=code
  &scope=openid profile email read:employees
  &redirect_uri=https://app.example.com/callback
  &state=xK9mN2pQ7rL4wV8t
  &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
  &code_challenge_method=S256
```

PKCE Flow

- Step 3: The authorization request
  - redirect_uri
    - Must exactly match a URI pre-registered with the AS
    - The AS will only redirect to registered URIs
    - This prevents an attacker registering a malicious redirect target
  - state
    - A random, per-request CSRF token generated by the client
    - Stored before the redirect
    - Verified on return.

```
https://auth.example.com/realms/hr-platform/protocol/openid-connect/auth
  ?client_id=hr-spa
  &response_type=code
  &scope=openid profile email read:employees
  &redirect_uri=https://app.example.com/callback
  &state=xK9mN2pQ7rL4wV8t
  &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
  &code_challenge_method=S256
```

PKCE Flow

- Step 3: The authorization request
  - code_challenge
    - The hash computed in step 2
  - code_challenge_method=S256
    - Tells the AS which hash algorithm was used

```
https://auth.example.com/realms/hr-platform/protocol/openid-connect/auth
  ?client_id=hr-spa
  &response_type=code
  &scope=openid profile email read:employees
  &redirect_uri=https://app.example.com/callback
  &state=xK9mN2pQ7rL4wV8t
  &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
  &code_challenge_method=S256
```

PKCE Flow

- Step 4: AS authenticates the user
  - This step is entirely the authorization server's responsibility.
  - Presents the login form
    - Username and password, passkey, social login, or whatever is configured
  - Enforces MFA if required by the realm or the requested scope
  - Applies step-up authentication rules if the requested scope demands higher assurance

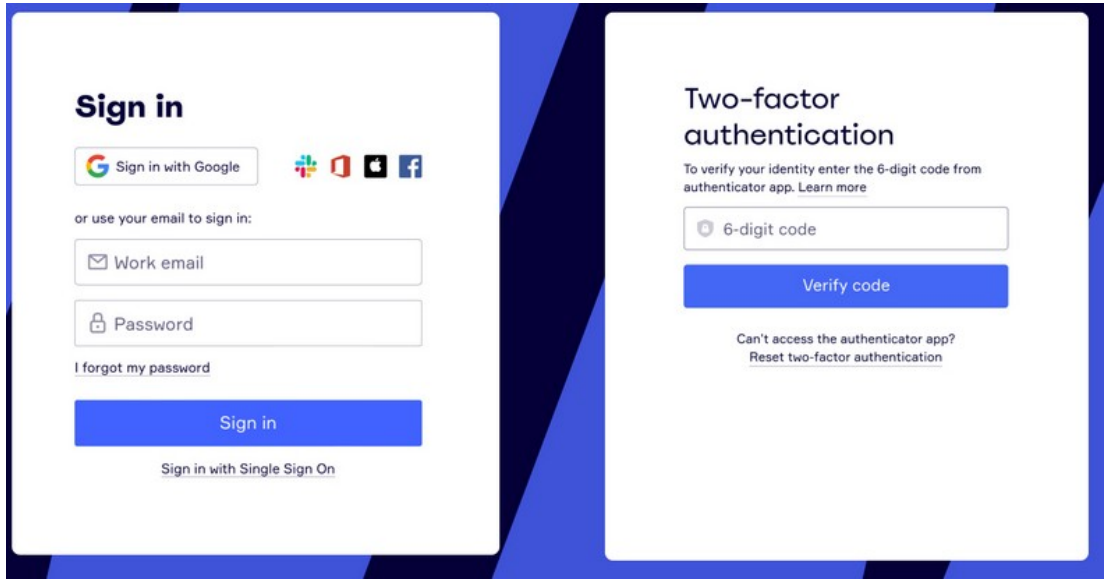


Image Credit: https://help.miro.com/hc/en-us/articles/7935469290002-Two-factor-authentication-2FA-user-guide-

PKCE Flow

- Step 4: AS authenticates the user
  - Presents the consent screen if the client is requesting scopes the user has not previously approved
  - Logs the authentication event for audit purposes
  - What the application never sees
    - The user's password
    - The MFA code
    - The authentication method used
    - Any intermediate authentication state

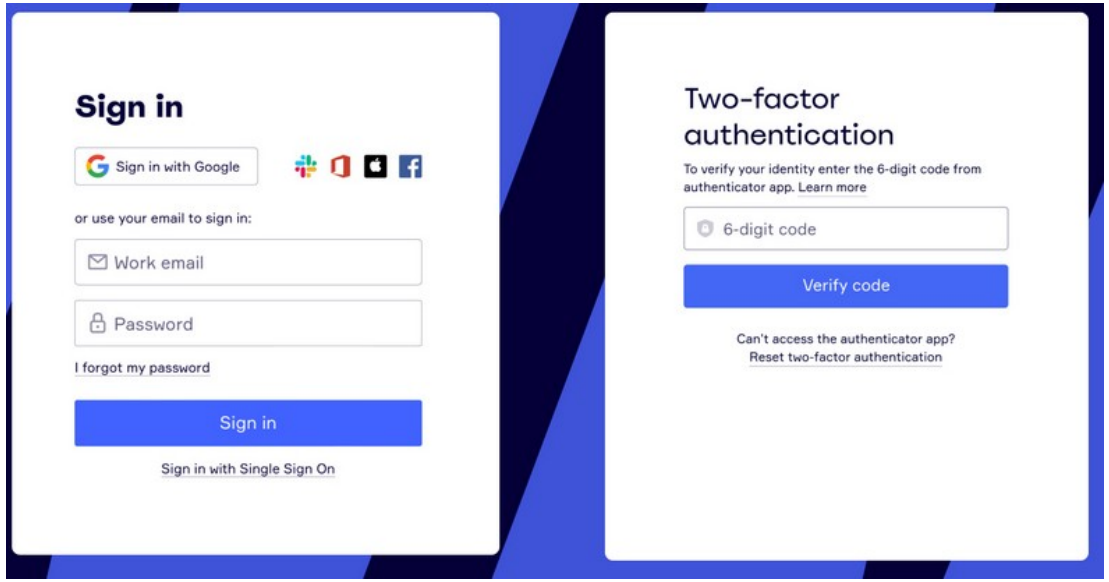


Image Credit: https://help.miro.com/hc/en-us/articles/7935469290002-Two-factor-authentication-2FA-user-guide-

PKCE Flow

- Step 4: AS authenticates the user
  - The consent screen is shown when the client requests scopes the user has not previously consented to
  - Lists the scopes in plain language
    - "This application wants to read your employee records"
  - The user can approve all, deny all, or (in some AS configurations) approve a subset
  - Consent decisions are stored by the AS and not re-requested on subsequent logins for the same client and scope combination

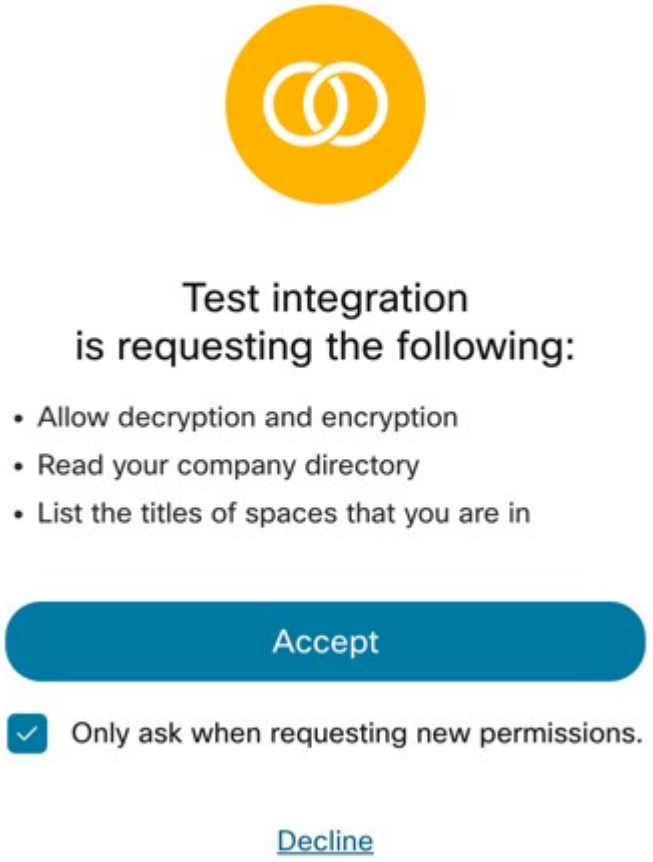


Image Credit: https://developer.webex.com/docs/run-an-oauth-integration

PKCE Flow

- Step 4: AS authenticates the user
  - The AS is the sole handler of credentials
  - The central security property that OAuth2 provides.
  - Delegating authentication entirely to the AS means a compromised application cannot steal credentials
  - And a user can authenticate using MFA or passkeys regardless of what the client application supports

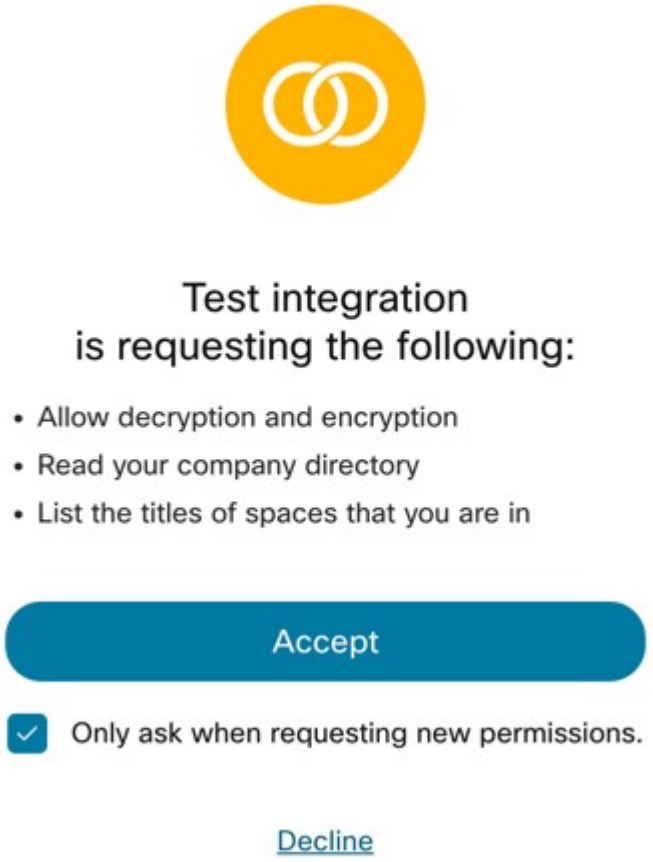


Image Credit: https://developer.webex.com/docs/run-an-oauth-integration

PKCE Flow

- Step 5: Authorization code returned
  - After successful authentication
  - AS redirects the browser back to the client's redirect_uri with two parameter

```
https://app.example.com/callback
  ?code=Splx1OBeZQQYbYS6WxSbIA
  &state=xK9mN2pQ7rL4wV8t
```

PKCE Flow

- Properties of the
  - Short-lived: typically valid for 60 to 300 seconds only
  - Single-use: the AS invalidates it immediately after the first token exchange attempt
  - Opaque: it is not a JWT but a random reference value that means nothing without the AS
  - Bound to the code_challenge stored by the AS when the authorization request was received

```
https://app.example.com/callback
  ?code=Splx1OBeZQQYbYS6WxSbIA
  &state=xK9mN2pQ7rL4wV8t
```

PKCE Flow

- The state verification
  - Must happen before anything else:
  - The client reads the state value from the redirect
  - Compares it against the value stored before the redirect in step 3
  - If they match: proceed to step 6
  - If they do not match: terminate the flow immediately
    - A mismatch means the redirect was not the result of the authorization request this client initiated
    - Do not proceed, do not log a warning, do not show a user-friendly error, terminate and require the user to start again

```
https://app.example.com/callback
  ?code=Splx1OBeZQQYbYS6WxSbIA
  &state=xK9mN2pQ7rL4wV8t
```

PKCE Flow

- Step 6: Exchanging code for tokens
  - The client now makes a direct, back-channel POST from the application server to the AS token endpoint
  - This call never goes through the browser
- What the AS does with this request
  - Looks up the stored code_challenge associated with this code
  - Applies BASE64URL(SHA256(code_verifier)) to the submitted verifier
  - Compares the result to the stored code_challenge
    - If they match: issues tokens
    - If they do not match: rejects the request

```
POST https://auth.example.com/realms/hr-platform/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=https://app.example.com/callback
&client_id=hr-spa
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

PKCE Flow

- Step 6: Exchanging code for tokens
- Why this stops the interception attack
  - An attacker who intercepted the code in step 5 can attempt this exchange
  - But they do not have the code_verifier
    - It was never in the browser URL or sent to the browser
    - Their exchange request fails at step 3 above
  - The legitimate client's exchange succeeds because it holds the code_verifier generated in step 1

```
POST https://auth.example.com/realms/hr-platform/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=https://app.example.com/callback
&client_id=hr-spa
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

PKCE Flow

- Step 7: Token response
  - A successful exchange returns the token set as shown
- Handling the tokens:
  - access_token is used for API calls, attached to every request
  - id_token is read by the client to establish user identity, not sent to the RS
  - refresh_token is stored securely
    - httpOnly cookie, never localStorage

```
{
  "access_token":  "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type":    "Bearer",
  "expires_in":    300,
  "id_token":      "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "8xLOxBtZp8...",
  "scope":         "openid profile email read:employees"
}
```

PKCE Flow

- Step 7: Token response
- Handling the tokens:
  - Never log the value of any token
    - They are credentials
  - The id_token and access_token are JWTs and are readable
    - They need to be verified before trusting

Token	Purpose	Lifetime	Contains
`access_token`	Presented to resource servers to authorise API calls	Short — 5 to 60 minutes	Scopes, roles, `sub` , `iss` , `aud` , `exp`
`id_token`	Proves to the client who the user is — OIDC identity assertion	Same as access token	`sub` , `name` , `email` , authentication time and method
`refresh_token`	Obtains new access tokens silently after expiry	Long — hours to days	Opaque reference — not a JWT

PKCE Flow

- Step 8: Calling the resource server
  - The token goes in the Authorization header with the Bearer scheme
    - Never send the token as a URL query parameter — ?token=eyJ...
    - URL query parameters appear in server access logs
    - They appear in browser history
  - They appear in Referer headers when the page links to external resources
  - They can be cached by proxies and CDNs
  - The RS reads the Authorization header and extracts the token and nothing else

```
GET /api/v1/employees
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

PKCE Flow

- Step 9: Resource server validation
  - The RS performs the checks shown on the right on every request, in order

Check	What is verified	Failure
Token present	`Authorization: Bearer` header is non-empty	401
Signature	Verified against current JWKS from the AS	401
Expiry (`exp`)	Current time is before `exp`, within clock skew tolerance	401
Issuer (`iss`)	Matches configured `issuer-uri`	401
Audience (`aud`)	Includes this RS's expected audience value	401
Scope	Token carries the scope required by this endpoint	403

Common Implementation Mistakes

Mistake	Consequence	Correct Approach
Storing `code_verifier` in localStorage	JavaScript-readable — defeats PKCE protection if XSS is present	Store in memory for the duration of the flow only
Reusing `code_verifier` across requests	A stolen challenge from a previous flow could be used to intercept a future one	Generate fresh per authorisation request
Using `code_challenge_method=plain`	The challenge equals the verifier — interception of the challenge reveals the verifier	Always use `S256`
Not verifying `state` on return	Login CSRF vulnerability — attacker can bind victim session to attacker account	Verify before any token exchange
Treating `state` mismatch as a warning	The flow is compromised — continuing exposes the user	Terminate immediately, require restart
Sending `access_token` in query parameter	Token appears in logs, history, and `Referer` headers	Always use `Authorization: Bearer` header
Storing `refresh_token` in localStorage	XSS-readable — long-lived credential exposed	Store in httpOnly, Secure, SameSite=Strict cookie
Not validating `aud` claim on the RS	Token issued for API A can be replayed against API B	Configure and enforce audience validation

Questions?